Introduction to

Deep Learning & Neural Networks with Keras

CONCEPTS • ARCHITECTURES • IMPLEMENTATION

Neural Networks | CNNs | RNNs | Transformers | Keras

> *Teaching machines to recognize patterns is the first step toward building intelligent systems.*

KASHIF MAQBOOL JOIYA
Data Scientist | AI Engineer

GitHub
https://github.com/KashifMaqbool

Table of Contents

Module 1: Introduction to Deep Learning and Neural Networks

1 Fundamentals of Deep Learning and Neural Networks with Keras

1.1 Biological Inspiration Behind Neural Networks

The module briefly introduces:

- Neurons in the human brain
- Biological neural networks

This helps in understanding how Artificial Neural Networks were inspired by the human brain.

1.2 Artificial Neural Networks

The course introduces the concepts associated with Artificial Neural Networks and explains how they function.

1.3 Forward Propagation

One of the first concepts covered is:

- Forward Propagation

This process explains how data moves through a neural network from input to output.

1.4 Learning Process in Neural Networks

The course also explains how Artificial Neural Networks learn from data.

Topics include:

- Gradient Descent
- Backpropagation
- Vanishing Gradient Problem
- Activation Functions

1.5 Deep Learning Libraries

The course introduces some of the most popular Deep Learning libraries, including:

- TensorFlow
- PyTorch
- Keras

1.5.1 Keras Library

Learners will use the Keras library to build models for:

- Regression
- Classification

1.5.2 Types of Neural Networks

The course compares:

- Shallow Neural Networks
- Deep Neural Networks

It also introduces:

- Supervised Deep Neural Networks
- Unsupervised Deep Neural Networks

1.5.3 Advanced Deep Learning Models

The course provides foundational knowledge of several advanced Deep Learning architectures.

1.5.4 Convolutional Neural Networks (CNNs)

Learners will understand the fundamentals of:

- Convolutional Neural Networks

The course also includes practical implementation using Keras.

1.5.5 Recurrent Neural Networks (RNNs)

The course introduces:

- Recurrent Neural Networks

These networks are commonly used for sequential and time-series data.

1.5.6 Transformers

Learners will study the fundamentals of:

- Transformers

These architectures are widely used in Natural Language Processing and Generative AI systems.

1.5.7 Autoencoders

The course also introduces:

- Autoencoders

These are neural networks used for representation learning and data compression.

- Fundamental Deep Learning concepts
- Neural Network architectures
- Popular Deep Learning libraries
- Model implementation using Keras

2 Introduction to Deep Learning

2.1 Overview

Deep learning is currently one of the hottest and most influential fields in data science. Over the last decade, deep learning has enabled the development of applications that were once considered nearly impossible. The rapid advancement of neural network architectures and computational power has led to remarkable breakthroughs across computer vision, speech processing, natural language processing, and artificial intelligence.

This lecture introduces several modern applications of deep learning and explains how neural networks form the foundation of these technologies.

2.2 Deep Learning Applications

2.2.1 Color Restoration

2.2.1.1 Introduction

One fascinating application of deep learning is automatic color restoration. In this task, a grayscale image is automatically converted into a colored image.

2.2.1.2 How It Works

Researchers in Japan developed a system using **Convolutional Neural Networks (CNNs)** that can:

- Analyze grayscale images
- Predict realistic colors
- Generate fully colored versions of black-and-white images

2.2.1.3 Significance

This application demonstrates how deep learning models can understand image content and generate visually realistic outputs automatically.

2.2.2 Speech Enactment

2.2.2.1 Introduction

Speech enactment is another advanced deep learning application in which:

- Audio is synthesized with video
- Lip movements are synchronized with speech
- Videos can appear completely realistic, even when generated artificially

2.2.2.2 Previous Challenges

Earlier systems attempting this task often produced results that appeared unnatural or uncanny.

2.2.2.3 Breakthrough Research

Researchers at the University of Washington developed a highly realistic system using:

- **Recurrent Neural Networks (RNNs)**
- Large video datasets
- Training data focused on a single individual

Their case study used videos of former U.S. President Barack Obama.

2.2.2.4 System Capabilities

The system can:

- Synchronize lip movements with synthesized audio
- Generate realistic speaking videos
- Extract audio from one video
- Apply that audio to another video while synchronizing lip movements

2.2.2.5 Example Demonstration

The system successfully generated realistic videos in which:

- Audio from one speech was synchronized with video from another speech
- Lip movements matched the spoken words accurately
- Many viewers could not distinguish the generated video from a real one

2.2.2.6 Importance

This application highlights the power of deep learning in:

- Video synthesis
- Facial motion generation
- Audio-visual synchronization
- Human-like media generation

2.2.3 Automatic Handwriting Generation

2.2.3.1 Introduction

Another fascinating application of deep learning is handwriting generation.

2.2.3.2 Research Contribution

Alex Graves at the University of Toronto developed an algorithm using **Recurrent Neural Networks (RNNs)** capable of generating realistic cursive handwriting.

2.2.3.3 Features of the System

The system allows users to:

- Input custom text
- Generate realistic handwritten output
- Select different handwriting styles
- Randomly generate handwriting styles automatically

2.2.3.4 Importance

This demonstrates how neural networks can learn:

- Writing patterns
- Pen movement dynamics
- Human handwriting styles

The generated handwriting closely resembles natural human cursive writing.

2.3 Additional Deep Learning Applications

2.3.1 Automatic Machine Translation

Deep learning models can automatically translate text between languages. Convolutional neural networks and other neural architectures are used to:

- Detect text in images
- Translate the detected text
- Display translations in real time

2.3.2 Sound Generation for Silent Movies

Deep learning can automatically add sounds to silent videos by:

- Analyzing visual scenes
- Searching databases of pre-recorded sounds
- Selecting audio that best matches the visual content

2.3.3 Object Classification and Detection

Deep learning is widely used for:

- Identifying objects in images
- Detecting multiple objects simultaneously
- Image recognition tasks

These applications are fundamental in computer vision systems.

2.3.4 Self-Driving Cars

Deep learning enables autonomous vehicles to:

- Detect roads and lanes
- Recognize traffic signs
- Identify pedestrians and obstacles
- Make driving decisions automatically

2.3.5 Chatbots

Modern chatbots use deep learning to:

- Understand human language
- Generate natural responses
- Simulate a human conversation

2.3.6 Text-to-Image Generators

Deep learning models can generate images directly from textual descriptions. These systems learn relationships between:

- Language
- Visual objects
- Artistic styles
- Scene composition

2.4 Neural Networks as the Foundation of Deep Learning

2.4.1 Neural Networks

Deep learning systems are built upon neural networks, which are computational models inspired by the human brain.

2.5 Types Mentioned in This Lecture

2.5.1 Convolutional Neural Networks (CNNs)

Used primarily for:

- Image processing
- Computer vision tasks
- Color restoration
- Object detection
- Image translation

2.5.2 Recurrent Neural Networks (RNNs)

Used primarily for:

- Sequential data processing
- Speech synthesis
- Handwriting generation
- Video and audio synchronization

2.6 Key Takeaways

2.6.1 Important Points

- Deep learning is one of the fastest-growing fields in data science.
- Neural networks are the foundation of deep learning applications.
- Convolutional Neural Networks are heavily used in image-related tasks.
- Recurrent Neural Networks are effective for sequence-based applications.
- Deep learning enables realistic media generation and intelligent automation.

2.7 Major Applications Covered

2.7.1 Image and Vision Applications

- Color restoration
- Object classification
- Object detection
- Text-to-image generation

2.7.2 Audio and Video Applications

- Speech enactment
- Lip synchronization
- Sound generation for silent videos

2.7.3 Language Applications

- Machine translation
- Chatbots

2.7.4 Creative Applications

- Automatic handwriting generation

2.7.5 Autonomous Systems

- Self-driving cars

2.8 Conclusion

Deep learning has transformed modern artificial intelligence by enabling machines to perform tasks that once required human intelligence. From image coloring and speech synchronization to handwriting generation and autonomous driving, deep learning continues to drive innovation across numerous industries.

As neural network architectures improve, the range and quality of deep learning applications will expand rapidly.

3 Neurons and Neural Networks

3.1 Introduction to Neurons and Neural Networks

Deep learning algorithms are largely inspired by the way neurons and neural networks function and process information in the brain.

One of the earliest drawings of a neuron was created in 1899 by **Santiago Ramon y Cajal**, who is now known as the father of modern neuroscience. His drawing was based on observations of a pigeon's brain under a microscope.

According to his observations:

- Neurons have large central bodies
- Long arms extend outward from the body
- These arms branch off and connect with other neurons
- Neural networks in the brain contain hundreds of thousands of tightly packed neurons

This biological structure inspired the design of artificial neural networks used in deep learning.

3.2 Structure of a Biological Neuron

3.2.1 Soma

The main body of the neuron is called the **soma**.

3.2.1.1 Functions of the Soma

- Contains the nucleus of the neuron
- Acts as the central processing area
- Receives information from dendrites
- Processes incoming electrical impulses

3.2.2 Dendrites

Dendrites are the extensive network of branches extending from the soma.

3.2.2.1 Functions of Dendrites

- Receive electrical impulses or signals
- Carry information from:
  - Sensors
  - Synapses of adjoining neurons
- Transfer the received information to the soma

3.2.3 Axon

The long arm extending from the soma is called the **axon**.

3.2.3.1 Functions of the Axon

- Carries processed information away from the soma
- Transfers signals to other neurons
- Connects the neuron to other neurons

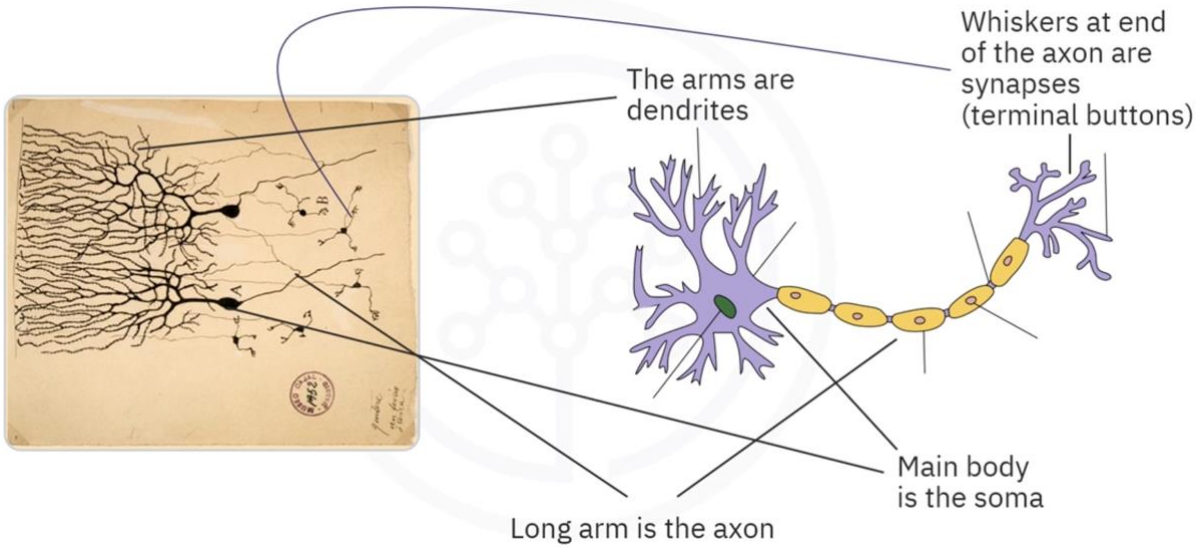
3.2.4 Synapses (Terminal Buttons)

The whisker-like endings at the end of the axon are called:

- Terminal buttons
- Synapses

3.2.4.1 Functions of Synapses

- Deliver output signals to other neurons
- Allow communication between neurons
- Pass processed information to thousands of connected neurons



3.3 How Information Flows Through a Neuron

3.3.1 Step 1: Receiving Information

- Dendrites receive electrical impulses
- Information comes from nearby neurons or external sensors

3.3.2 Step 2: Sending Information to the Soma

- Dendrites carry the received impulses to the soma

3.3.3 Step 3: Processing Information

Inside the nucleus and soma:

- Electrical impulses are combined
- Information is processed
- A decision/output signal is generated

3.3.4 Step 4: Passing Information Through the Axon

- The processed information moves through the axon

3.3.5 Step 5: Communicating with Other Neurons

- Synapses transmit the output signal
- The output of one neuron becomes the input to many other neurons

3.4 Learning in the Brain

Learning occurs in the brain through repeated activation of neural connections.

3.4.1 Key Concepts of Biological Learning

- Some neural connections are activated more frequently than others
- Frequently used connections become stronger over time
- Stronger neural connections are more likely to produce desired outcomes
- Reinforced connections improve learning efficiency

This strengthening process is the biological inspiration behind learning in artificial neural networks.

3.5 Artificial Neurons

Artificial neurons are designed to behave similarly to biological neurons.

3.5.1 Components of an Artificial Neuron

Artificial neurons also contain structures analogous to:

- Soma
- Dendrites
- Axon

3.5.2 Similarities Between Biological and Artificial Neurons

Biological Neuron	**Artificial Neuron**
Receives inputs through dendrites	Receives input data
Processes information in the soma	Processes data mathematically
Sends outputs through the axon	Produces output to other neurons
Learns by strengthening connections	Learns by adjusting weights

3.5.3 Neural Connections

- Artificial neurons can connect to many other neurons
- Outputs from one neuron become inputs to other neurons
- This interconnected structure forms an artificial neural network

3.6 Deep Learning and Brain Inspiration

Deep learning systems are inspired by:

- Biological neurons
- Neural communication mechanisms
- Brain learning processes
- Strengthening of neural connections

Artificial neural networks imitate these biological processes to learn patterns from data and make predictions.

3.7 Summary

3.7.1 Important Terms

Term	Description
Soma	Main body of the neuron containing the nucleus
Dendrites	Branch-like structures that receive signals
Axon	Long structure that carries processed signals
Synapses	End points that transfer signals to other neurons

3.7.2 Key Points

- Deep learning is inspired by biological neural networks
- Neurons communicate using electrical impulses
- Dendrites receive information
- The soma processes information
- The axon carries processed information
- Synapses transfer outputs to other neurons
- Learning occurs by strengthening frequently used neural connections
- Artificial neurons mimic biological neuron behavior

4 Artificial Neural Networks

4.1 Introduction to Artificial Neural Networks

Artificial Neural Networks (ANNs) are inspired by the structure and functioning of biological neurons in the human brain. Artificial neurons are designed to mimic the behavior of real neurons by receiving inputs, processing them, and producing outputs.

One of the earliest artificial neurons was designed to:

- Accept binary inputs
- Produce a binary output

An artificial neuron is also known as a **Perceptron**.

4.2 Perceptron (Artificial Neuron)

A perceptron performs the following operations:

- Calculates the weighted sum of inputs
- Compares the result with a threshold
- Produces an output:
    - 1 if the sum exceeds the threshold
    - 0 otherwise

A perceptron is the fundamental building block of artificial neural networks.

4.3 Structure of a Neural Network

In a neural network, multiple perceptrons are arranged into layers.

4.3.1 Input Layer

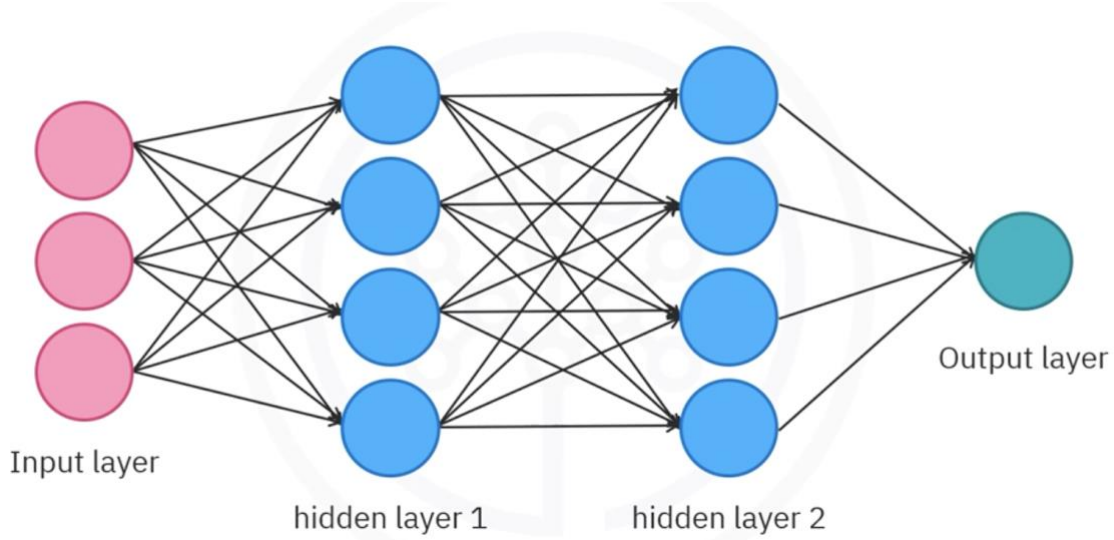
The first layer that feeds data into the network is called the **Input Layer**.

4.3.2 Hidden Layers

The layers between the input and output layers are called **Hidden Layers**.

4.3.3 Output Layer

The layer responsible for generating the final output is called the **Output Layer**.



4.4 Main Topics in Neural Networks

When working with neural networks, three major concepts are commonly discussed:

- Forward Propagation
- Backpropagation
- Activation Functions

This lecture primarily focuses on **Forward Propagation**.

4.5 Forward Propagation

Forward propagation is the process through which data moves through the layers of a neural network:

- Data flows from the input layer
- Passes through hidden layers
- Reaches the output layer

The flow of information occurs through connections between neurons.

4.6 Mathematical Formulation of a Neuron

Consider a neuron with two inputs:

- x_1
- x_2

Each input is associated with a weight:

- w_1
- w_2

The neuron computes a weighted sum of the inputs and adds a bias term.

4.6.1 Linear Combination Formula

The linear combination is represented as:

$$z = w_1 x_1 + w_2 x_2 + b$$

Where:

- z= linear combination of inputs and weights
- b= bias
- a= output of the neuron

4.6.2 Important Notation

Throughout neural network discussions:

- $'z'$ always represents the linear combination
- $'a'$ always represents the neuron's output

4.7 Importance of Activation Functions

Simply producing a weighted sum limits the capability of neural networks.

To solve complex problems, neural networks apply a nonlinear transformation using an **Activation Function**.

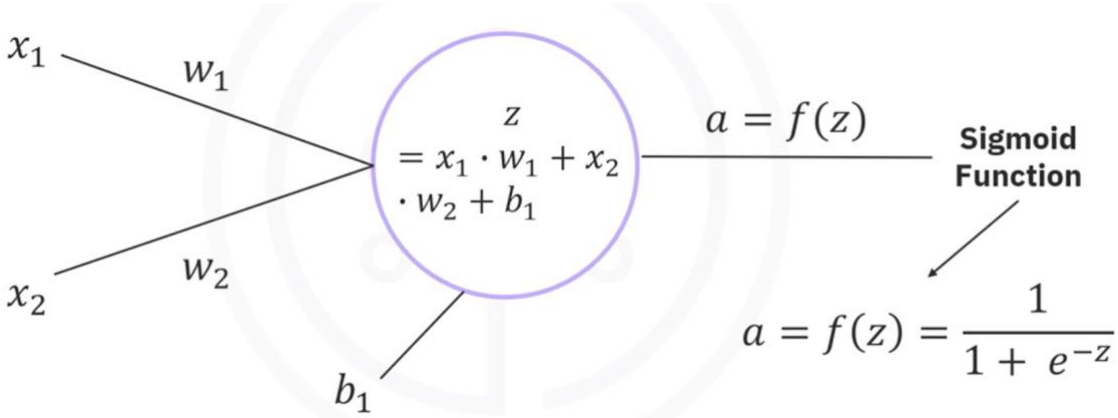
4.7.1 Sigmoid Activation Function

One of the most popular activation functions is the **Sigmoid Function**.

$$a = \sigma(z) = \frac{1}{1 + e^{-z}}$$

4.7.2 Behavior of the Sigmoid Function

- If z is a very large positive number:
  - Output approaches 1
- If z is a very large negative number:
  - Output approaches 0



4.8 Role of Activation Functions

Activation functions determine whether a neuron should be activated.

They help the network decide:

- Which information is relevant
- Which information should be ignored

4.8.1 Key Takeaway

Without activation functions, a neural network behaves similarly to a linear regression model.

Activation functions enable neural networks to perform complex nonlinear tasks such as:

- Image classification
- Language translation
- Pattern recognition

4.9 Example: Forward Propagation with One Neuron

Consider a neural network with:

- One input neuron
- One perceptron

4.9.1 Given Values

Parameter	Value
x_1	0.1
w_1	0.15
b_1	0.4

4.9.2 Step 1: Compute the Linear Combination

Using the formula:

$$z = w_1 x_1 + b_1$$

Substituting the values:

$$z = (0.15)(0.1) + 0.4$$
$$z = 0.015 + 0.4$$
$$z = 0.415$$

4.9.3 Step 2: Apply the Sigmoid Activation Function

$$a = \sigma(0.415)$$

Output:

$$a = 0.6023$$

Therefore, the neuron outputs:

$$0.6023$$

4.10 Example: Neural Network with Two Neurons

In a network with two neurons:

- The output of the first neuron becomes the input to the second neuron

4.10.1 Process

The second neuron:

1. Takes a_1 as input
2. Multiply it by weight w_2

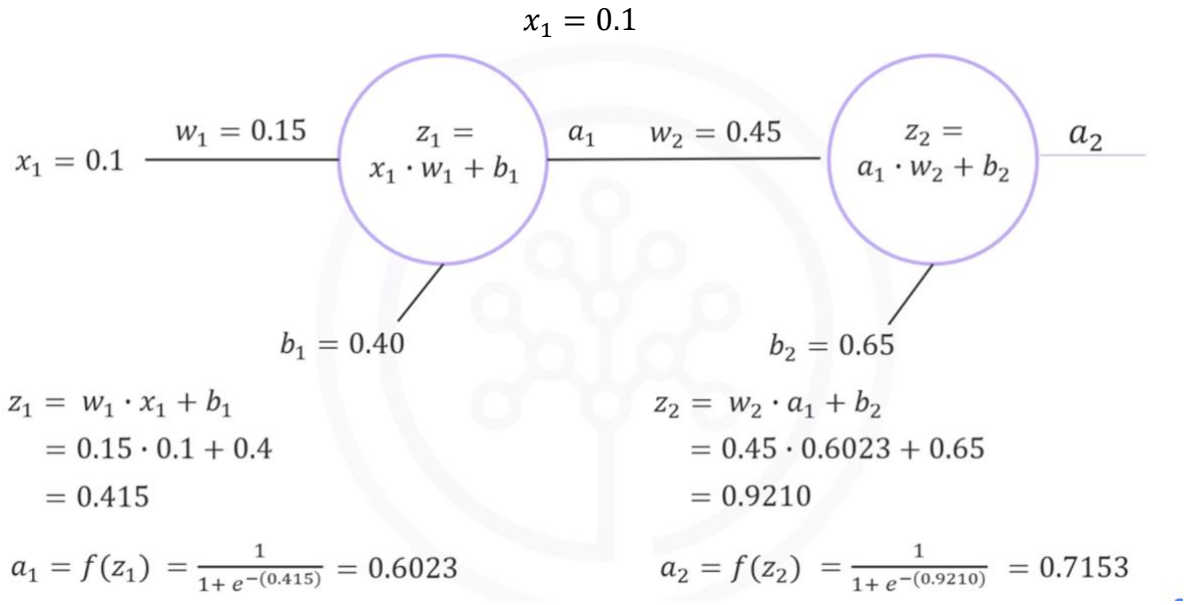
3. Adds bias b_2
4. Applies the sigmoid activation function

4.10.2 Final Output

The final predicted output becomes:

$$0.7153$$

This is the predicted value for the original input:



4.11 How Neural Networks Make Predictions

Regardless of how large or complex a neural network becomes, the prediction process follows the same sequence:

1. Compute weighted sums
2. Add biases
3. Apply activation functions
4. Pass outputs to the next layer
5. Generate final prediction

4.12 Summary

4.12.1 Neural Network Layers

Layer Type	Description
Input Layer	Feeds input data into the network
Hidden Layer	Intermediate processing layers
Output Layer	Produces the final output

4.12.2 Important Concepts

Concept	Description
Forward Propagation	Process of passing data from input to output
Weights	Control the influence of inputs
Bias	Constant added to the weighted sum
Activation Function	Applies nonlinear transformation
Sigmoid Function	Common activation function producing values between 0 and 1

4.12.3 Final Takeaway

Given:

- Inputs
- Weights
- Biases
- Activation functions

You should be able to compute the output of a neural network for any input using forward propagation.

5 Deep Learning and Artificial Neural Networks — Lesson Summary

5.1 Introduction to Deep Learning

Deep learning is one of the hottest and most important subjects in data science. It enables machines to learn complex patterns and perform tasks that traditionally required human intelligence.

5.2 Applications of Deep Learning

5.2.1 Color Restoration Applications

Color restoration systems can automatically convert grayscale images into fully colored images.

5.2.2 Speech Enactment Applications

Speech enactment applications can:

- Synthesize audio clips with lip movements in videos
- Extract audio from one video
- Synchronize lip movements with audio from another video

5.2.3 Handwriting Generation Applications

Handwriting generation systems can rewrite provided text in highly realistic cursive handwriting across a wide variety of writing styles.

5.3 Biological Inspiration Behind Deep Learning

Deep learning algorithms are largely inspired by the way biological neurons and neural networks function in the human brain.

5.4 Structure of a Biological Neuron

5.4.1 Soma

The main body of a neuron is called the **Soma**.

5.4.2 Dendrites

The extensive network of arm-like structures extending from the soma are called **Dendrites**.

Functions of dendrites:

- Receive electrical impulses
- Carry information from neighboring neurons
- Transfer impulses to the soma

5.4.3 Axon

The long extension projecting from the soma is called the **Axon**.

Function of the axon:

- Carries processed information away from the neuron

5.4.4 Synapses

The whisker-like endings at the end of the axon are called **Synapses**.

Functions of synapses:

- Transmit signals to adjoining neurons
- Allow communication between neurons

5.5 Information Processing in Biological Neurons

The processing workflow inside a neuron is as follows:

1. Dendrites receive electrical impulses

2. Signals are passed to the soma
3. The nucleus processes and combines the impulses
4. The processed information is transferred to the axon
5. The axon carries information to the synapses
6. The output becomes input for thousands of other neurons

5.6 Learning Mechanism in the Brain

Learning in the human brain occurs through repeated activation of certain neural connections.

Repeated activation strengthens and reinforces these neural pathways over time.

5.7 Artificial Neurons

Artificial neurons are designed to behave similarly to biological neurons.

They form the fundamental building blocks of artificial neural networks.

5.8 Neural Network Layers

5.8.1 Input Layer

The first layer that feeds input data into the neural network is called the **Input Layer**.

5.8.2 Hidden Layers

The intermediate layers between the input and output layers are called **Hidden Layers**.

5.8.3 Output Layer

The layer responsible for generating the final network output is called the **Output Layer**.

5.9 Forward Propagation

Forward propagation is the process through which data moves through the layers of a neural network:

- From the input layer
- Through hidden layers
- To the output layer

This process enables the network to generate predictions or outputs.

5.10 Weights and Biases in Neural Networks

Neural networks use:

- Weights

- Biases

to process input data and compute outputs.

Given a neural network with known weights and biases, it is possible to compute the output for any given input.

5.11 Key Takeaways

Concept	Description
Deep Learning	A major field in data science inspired by the human brain
Artificial Neuron	Computational model inspired by biological neurons
Dendrites	Receive information from other neurons
Soma	Main body of the neuron
Axon	Transfers processed information
Synapses	Connect neurons for communication
Input Layer	Receives input data
Hidden Layers	Perform intermediate computations
Output Layer	Produces final predictions
Forward Propagation	Flow of data through the network
Weights and Biases	Parameters used to compute outputs

Module 2: Basics of Deep Learning

6 Gradient Descent

6.1 Introduction to Gradient Descent

Before learning how neural networks optimize weights and biases, it is important to understand Gradient Descent. Gradient Descent is an iterative optimization technique used to minimize a function, especially in machine learning models.

6.2 Problem Setup: Dataset and Objective

Consider a simple dataset shown as a scatterplot.

For simplicity:

- The dataset is constructed such that **z = 2x**

We want to find a weight value **w** such that a line:

- z = wx

best fits the data.

The goal is to determine the optimal value of **w** that minimizes prediction error.

6.3 Cost (Loss) Function

To measure how well a chosen value of **w** fits the data, we define a cost function:

Cost Function (J):

- It computes the difference between actual values (z) and predicted values (wx)
- It squares these differences
- It sums them across all data points

Mathematically:

$J(w) = \Sigma (z - wx)^2$

6.3.1 Purpose of the Cost Function

- Measures prediction error
- Helps evaluate model performance
- The smaller the value, the better the fit

The best value of **w** is the one that minimizes J(w).

6.4 Shape of the Cost Function

The cost function in this example has a special property:

- It forms a **parabola**
- It has a **single global minimum**
- It has a **unique optimal solution**

For this dataset:

- The minimum occurs at: **w = 2**
- This produces the perfect model: **z = 2x**

At this point, the line fits all data points exactly.

6.5 Why Optimization is Needed

In real-world problems:

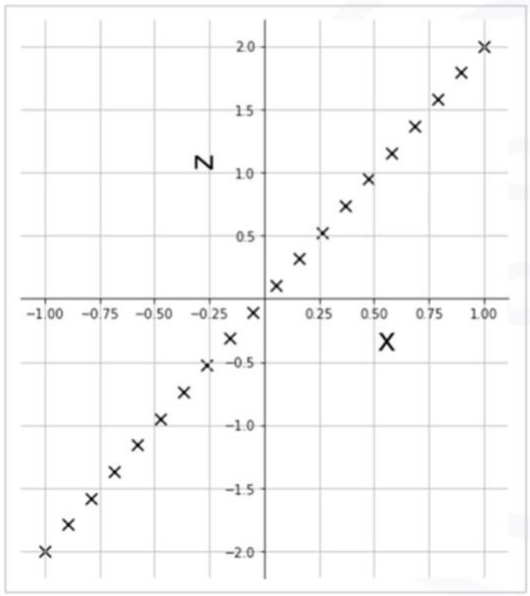
- There are multiple variables (features)
- There are many weights to optimize
- The cost function cannot be visualized easily

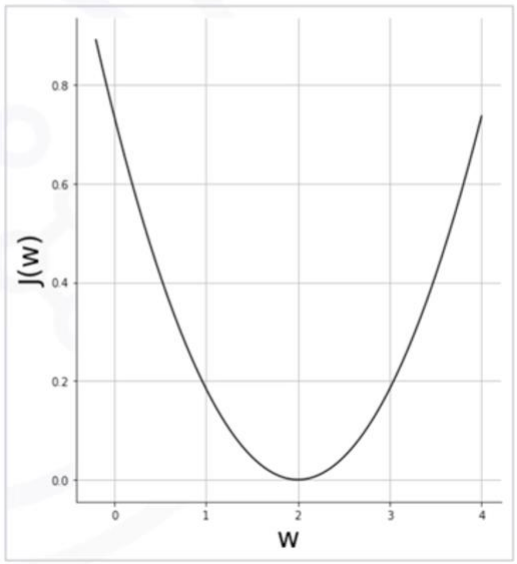
Therefore, we need an automatic method to find the minimum.

This method is called **Gradient Descent**.

$$Z = wX$$

$$J(w) = \frac{1}{2m}\sum_{i=1}^{m}(z_i - wx_i)^2$$





6.6 Gradient Descent Algorithm

Gradient Descent is an iterative optimization algorithm used to find the minimum of a function.

6.6.1 Core Idea

At each step:

- Compute the gradient (slope) of the cost function
- Move in the direction opposite to the gradient
- Repeat until reaching the minimum

6.7 Initialization

The process starts by choosing a random initial value of w:

Example:

- $w_0 = 0.2$

This is the starting point of optimization.

6.8 Gradient Calculation

At each step:

- Compute the slope of the cost function at the current value of w
- This slope is called the **gradient**

The gradient indicates:

- Direction of steepest increase

So we move in the opposite direction to reduce error.

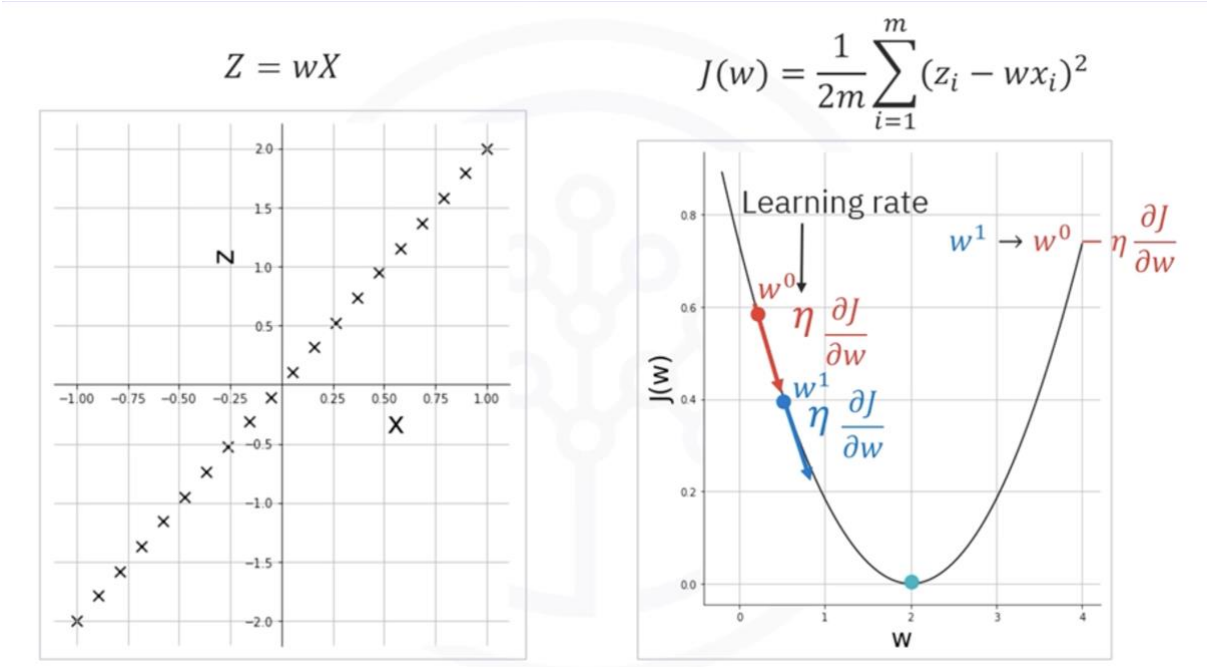
6.9 Weight Update Rule

The weight is updated using:

$w_1 = w_0 - \eta \times \text{gradient}$

Where:

- w_0 = current weight
- w_1 = updated weight
- η = learning rate
- gradient = slope of the cost function



6.10 Iterative Process

Gradient Descent repeats the following steps:

- Compute the gradient at the current point
- Update the weight using the rule
- Move closer to the minimum
- Repeat until convergence

6.10.1 Stopping Condition

The algorithm stops when:

- It reaches the minimum, OR
- The change becomes very small (below a threshold)

6.11 Learning Rate

The learning rate (η) controls the step size during updates.

6.11.1 Large Learning Rate

- Takes large steps
- May overshoot the minimum
- Can miss the optimal solution

6.11.2 Small Learning Rate

- Takes very small steps
- Requires many iterations
- Slow convergence

6.11.3 Key Insight

Choosing the learning rate is critical for successful optimization.

6.12 Example: Gradient Descent Iterations (Learning Rate = 0.4)

6.12.1 Initial State

- w = 0
- z = 0

This produces a horizontal line.

6.12.2 Observation

- The line is a poor fit
- The cost is very high

6.13 First Iteration

- The gradient is very steep at w = 0
- Large update step occurs
- w moves closer to 2

Result:

- Significant reduction in cost
- Better fitting line

6.14 Second Iteration

- Slope becomes less steep
- Step size becomes smaller
- w continues moving toward 2

Result:

- Cost decreases further
- Fit improves

6.15 Third Iteration

- Gradient continues to reduce
- Smaller updates occur
- w approaches optimal value

6.16 Fourth Iteration

- w is very close to 2
- Line nearly fits the data perfectly

Final observation:

- Model is almost fully optimized

6.17 Key Insight from Iterations

Each update follows this principle:

- Move opposite to the gradient direction
- Reduce cost step by step
- Converge gradually toward the minimum

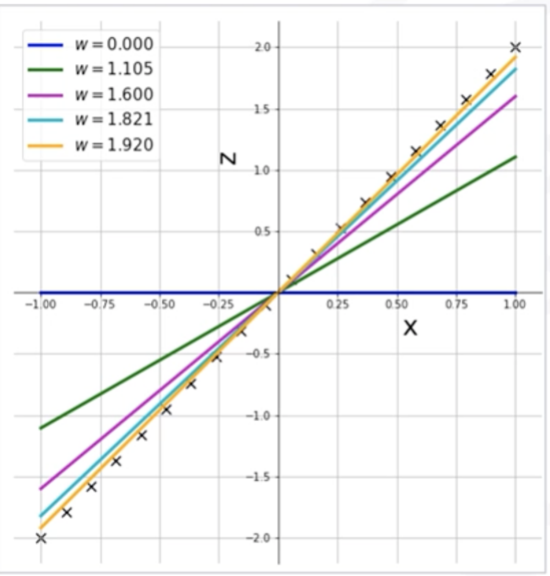
6.18 Direction of Movement

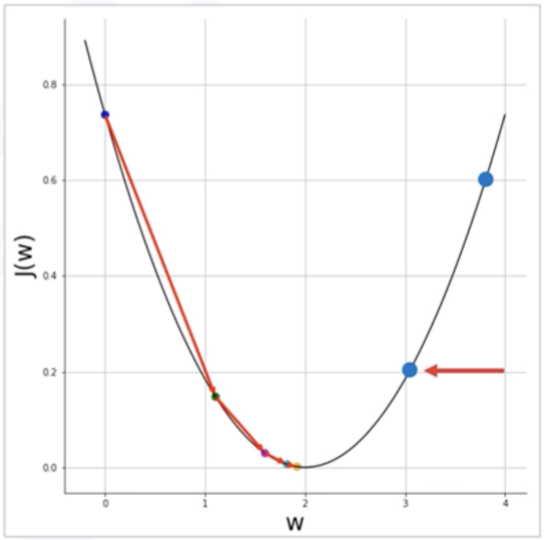
- If w is to the right of the minimum → gradient is positive → move left
- If w is to the left of the minimum → gradient is negative → move right

Thus, Gradient Descent always moves toward the minimum.

$$Z = wX$$

$$J(w) = \frac{1}{2m}\sum_{i=1}^{m}(z_i - wx_i)^2$$





$$w^{i+1} \rightarrow w^{i} - \eta \frac{\partial J}{\partial w}$$

6.19 Key Takeaways

- Gradient Descent is an iterative optimization algorithm
- It minimizes a cost function step by step
- The cost function measures prediction error
- The gradient determines the direction of movement
- The learning rate controls the step size
- Proper learning rate selection is essential for convergence
- Repeated iterations gradually improve model fit until optimal performance is reached

7 Neural Network Training and Backpropagation

7.1 Introduction to Neural Network Training

Neural networks learn by training and optimizing their weights and biases. During training, the network adjusts these parameters to minimize prediction error and improve accuracy.

7.2 Supervised Learning

Neural network training is commonly performed using supervised learning.

7.2.1 Definition

In supervised learning:

- Each input data point has a corresponding label or ground truth
- The network learns by comparing predictions with actual values

7.2.2 Training Objective

The goal is to minimize the difference between:

- Predicted output
- Ground truth (target output)

7.3 Neural Network Training Process

Training begins when:

- The network prediction does not match the ground truth

The network then:

1. Computes the prediction error
2. Propagates the error backward
3. Updates weights and biases
4. Repeats the process iteratively

7.4 Forward Propagation Review

Before training, the network performs forward propagation.

7.4.1 Forward Propagation Steps

- Input passes through neurons
- Weighted sums are calculated
- Activation functions are applied
- Final prediction is generated

The predicted output is represented as:

$$a_2$$

The ground truth is represented as:

$$T$$

7.5 Error (Loss) Function

The network calculates the error between:

- Ground truth T
- Predicted value a_2

7.5.1 Squared Error Function

$$E = \frac{1}{2}(T - a_2)^2$$

This error function represents the cost or loss function.

7.6 Mean Squared Error (MSE)

In real-world training:

- Networks are trained on thousands of data points
- Error is averaged across all samples

7.6.1 Mean Squared Error Formula

$$MSE = \frac{1}{N}\sum(T - a_2)^2$$

Where:

- N= number of training samples

7.7 Backpropagation

Backpropagation is the process of propagating the error backward through the network to update weights and biases.

7.7.1 Purpose of Backpropagation

Backpropagation helps:

- Compute gradients
- Determine how much each weight contributed to the error
- Update weights to minimize loss

7.8 Updating Weights Using Gradient Descent

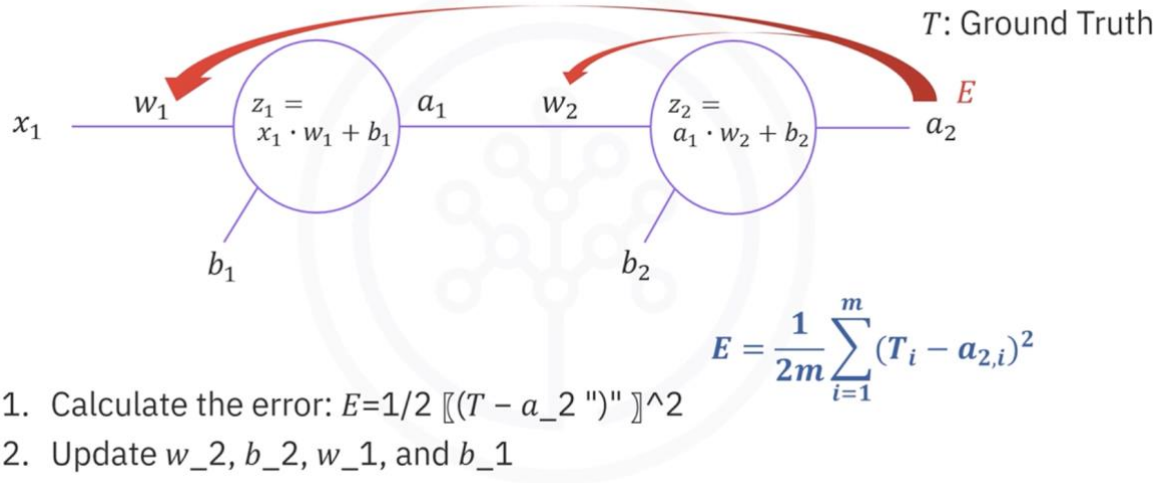
Weights are updated using Gradient Descent.

7.8.1 General Weight Update Rule

$$w_{new} = w_{old} - \eta \frac{\partial E}{\partial w}$$

Where:

- η= learning rate
- $\frac{\partial E}{\partial w}$= gradient of error with respect to weight



7.9 Chain Rule in Backpropagation

The error is not directly dependent on all weights.

Therefore, we use the Chain Rule from calculus.

7.10 Updating Weight w_2

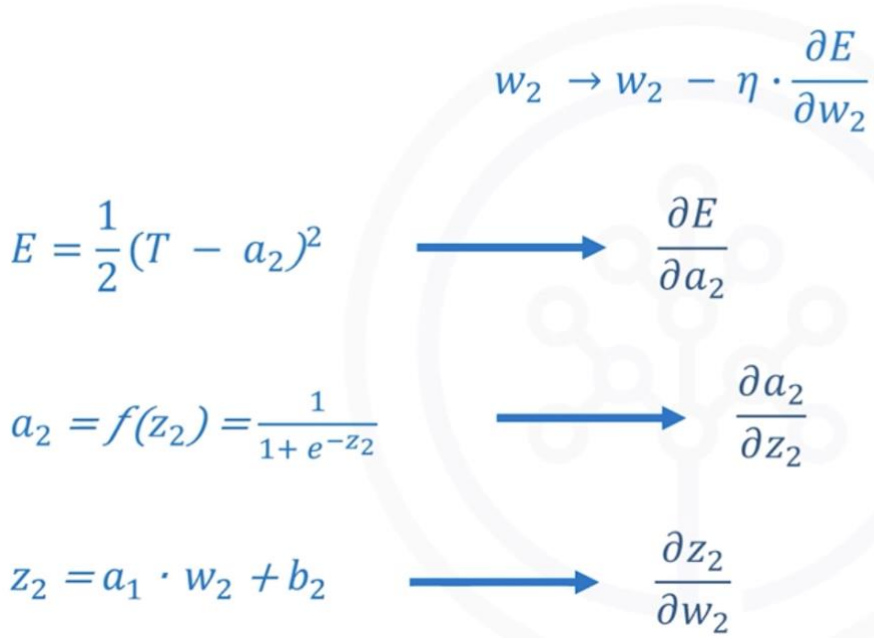
7.10.1 Dependency Chain

The error depends on:

$$E \rightarrow a_2 \rightarrow z_2 \rightarrow w_2$$

7.10.2 Chain Rule Expression

$$\frac{\partial E}{\partial w_2} = \frac{\partial E}{\partial a_2} \cdot \frac{\partial a_2}{\partial z_2} \cdot \frac{\partial z_2}{\partial w_2}$$



7.10.3 Individual Derivatives

7.10.3.1 Derivative of Error with Respect to a_2

$$\frac{\partial E}{\partial a_2} = -(T - a_2)$$

7.10.3.2 Derivative of Activation with Respect to z_2

$$\frac{\partial a_2}{\partial z_2} = a_2(1 - a_2)$$

7.10.3.3 Derivative of $z_2$with Respect to w_2

$$\frac{\partial z_2}{\partial w_2} = a_1$$

7.10.4 Final Gradient Expression for w_2

$$\frac{\partial E}{\partial w_2} = -(T - a_2) \cdot a_2(1 - a_2) \cdot a_1$$

7.11 Updating Bias b_2

The update process for b_2 is similar.

7.11.1 Important Difference

$$\frac{\partial z_2}{\partial b_2} = 1$$

Instead of:

$$\frac{\partial z_2}{\partial w_2} = a_1$$

So,

$$\frac{\partial E}{\partial b_2} = -(T - a_2) \cdot a_2(1 - a_2) \cdot 1$$

7.12 Updating Weight w_1

7.12.1 Dependency Chain

The error depends on:

$$E \to a_2 \to z_2 \to a_1 \to z_1 \to w_1$$

7.12.2 Chain Rule Expression

$$\frac{\partial E}{\partial w_1} = \frac{\partial E}{\partial a_2} \cdot \frac{\partial a_2}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1}$$

7.12.3 Individual Derivatives

7.12.3.1 Output Error Derivative

$$\frac{\partial E}{\partial a_2} = -(T - a_2)$$

7.12.3.2 Activation Derivative at Output Layer

$$\frac{\partial a_2}{\partial z_2} = a_2(1 - a_2)$$

7.12.3.3 Derivative of z_2 with Respect to a_1

$$\frac{\partial z_2}{\partial a_1} = w_2$$

7.12.3.4 Hidden Layer Activation Derivative

$$\frac{\partial a_1}{\partial z_1} = a_1(1 - a_1)$$

7.12.3.5 Derivative of z_1 with Respect to w_1

$$\frac{\partial z_1}{\partial w_1} = x_1$$

7.12.4 Final Gradient Expression for w_1

$$\frac{\partial E}{\partial w_1} = -(T - a_2) \cdot a_2(1 - a_2) \cdot w_2 \cdot a_1(1 - a_1) \cdot x_1$$

7.13 Updating Bias b_1

The same backpropagation approach is used to compute the update rule for b_1.

So,

$$\frac{\partial E}{\partial b_1} = -(T - a_2) \cdot a_2(1 - a_2) \cdot w_2 \cdot a_1(1 - a_1) \cdot 1$$

7.14 Example of Backpropagation

7.15 Initial Network Parameters

Consider a neural network with:

- Two neurons
- One input
- Initial weights and biases

7.16 Forward Propagation Results

After forward propagation:

7.16.1 Hidden Layer

$$z_1 = 0.415$$
$$a_1 = 0.6023$$

7.16.2 Output Layer

$$z_2 = 0.9210$$
$$a_2 = 0.7153$$

The predicted output is:

$$0.7153$$

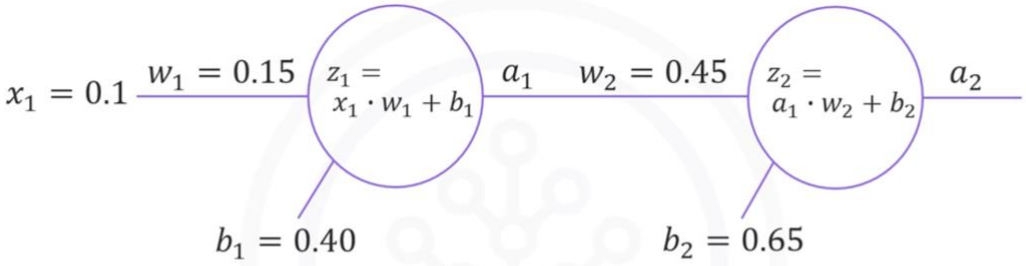
For input:

$$x = 0.1$$

7.17 Ground Truth

Assume the actual target value is:

$$T = 0.25$$



$$z_1 = w_1 \cdot x_1 + b_1$$
$$= 0.15 \cdot 0.1 + 0.4$$
$$= 0.415$$

$$a_1 = f(z_1) = \frac{1}{1+e^{-(0.415)}} = 0.6023$$

$$z_2 = w_2 \cdot a_1 + b_2$$
$$= 0.45 \cdot 0.6023 + 0.65$$
$$= 0.9210$$

$$a_2 = f(z_2) = \frac{1}{1+e^{-(0.9210)}} = 0.7153$$

7.18 Error Computation

The network first computes the error between:

- Predicted value
- Ground truth

7.19 Training Iterations (Epochs)

The network updates weights and biases for:

- A predefined number of epochs (e.g., 1000)
- Or until the error becomes very small

Example threshold:

$$0.001$$

7.20 Learning Rate

Assume:

$$\eta = 0.4$$

7.21 First Iteration Updates

7.21.1 Updating w_2

Calculated gradient:

$$\frac{\partial E}{\partial w_2} = 0.05706$$

Updated weight:

$$w_2 = 0.427$$

$$w_2 \rightarrow w_2 - \eta \cdot \big(-(T - a_2)\big) \cdot \big(a_2(1 - a_2)\big) \cdot (a_1)$$

$$w_2 \rightarrow 0.45 - 0.4 \cdot \big(-(0.25 - 0.7153)\big) \cdot \big(0.7153(1 - 0.7153)\big) \cdot (0.6023)$$

$$w_2 \rightarrow 0.45 - 0.4 \cdot 0.05706$$

$$w_2 \rightarrow 0.427$$

7.22 Updating b_2

Calculated gradient:

$$\frac{\partial E}{\partial b_2} = 0.0948$$

Updated bias:

$$b_2 = 0.612$$

7.23 Updating w_1

Calculated gradient:

$$\frac{\partial E}{\partial w_1} = 0.001021$$

Updated weight:

$$w_1 = 0.1496$$

7.24 Updating b_1

Calculated gradient:

$$\frac{\partial E}{\partial b_1} = 0.01021$$

Updated bias:

$$b_1 = 0.3959$$

This completes one iteration (epoch) of training.

7.25 Repeating the Training Process

After updating the parameters:

1. Perform forward propagation again
2. Compute the new prediction
3. Calculate new error
4. Apply backpropagation
5. Update weights and biases again

This cycle repeats continuously.

7.26 Complete Neural Network Training Algorithm

7.26.1 Step 1 — Initialize Parameters

Initialize:

- Weights
- Biases

Using random values.

7.27 Step 2 — Forward Propagation

Compute network predictions using forward propagation.

7.28 Step 3 — Compute Error

Calculate the difference between:

- Predicted output
- Ground truth

7.29 Step 4 — Backpropagation

Propagate the error backward through the network.

7.30 Step 5 — Update Parameters

Update:

- Weights
- Biases

Using Gradient Descent.

7.31 Step 6 — Repeat

Repeat all steps until:

- Maximum epochs are reached
- Or an error falls below a threshold

7.32 Key Concepts Summary

7.32.1 Forward Propagation

Forward propagation:

- Computes predictions
- Passes information from input to the output layer

7.32.2 Backpropagation

Backpropagation:

- Propagates error backward
- Computes gradients
- Helps optimize parameters

7.32.3 Gradient Descent

Gradient Descent:

- Minimizes error
- Updates weights iteratively

7.32.4 Learning Rate

The learning rate:

- Controls update size
- Affects convergence speed

7.32.5 Epoch

An epoch is:

- One complete training cycle
- One forward pass plus one backward pass

7.33 Final Takeaways

- Neural networks learn by minimizing prediction error.
- Training uses supervised learning with labeled data.
- Forward propagation computes predictions.
- Backpropagation computes gradients using the Chain Rule.
- Gradient Descent updates weights and biases.
- Training repeats iteratively until convergence.
- Proper selection of the learning rate is essential for stable learning.

8 Vanishing Gradient Problem

8.1 Introduction to the Vanishing Gradient Problem

The Vanishing Gradient Problem is one of the major challenges encountered during the training of deep neural networks.

8.2 Background

Early neural networks commonly used the sigmoid activation function.

Although sigmoid functions helped introduce non-linearity into neural networks, they also created a serious optimization issue known as the Vanishing Gradient Problem.

This problem significantly slowed the development and success of deep neural networks.

8.3 Sigmoid Activation Function

The sigmoid activation function produces outputs between 0 and 1.

The sigmoid equation is:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

8.3.1 Characteristics of the Sigmoid Function

- Output values always lie between 0 and 1

- Smooth and differentiable
- Commonly used in early neural networks

However, despite these advantages, sigmoid functions create very small gradients during backpropagation.

8.4 Understanding the Vanishing Gradient Problem

During training, neural networks use backpropagation to update weights.

Backpropagation computes gradients of the error with respect to each weight in the network.

8.4.1 Problem Observation

In a simple network with only two neurons:

- Gradients already become very small
- Earlier layer gradients are even smaller

Especially:

- The gradient of the error with respect to W_1 becomes extremely small

8.5 Why Gradients Vanish

Because sigmoid outputs are between 0 and 1:

$$0 < \sigma(x) < 1$$

During backpropagation:

- Many small values are multiplied together
- Multiplying numbers less than 1 repeatedly causes the result to shrink rapidly

8.5.1 Example

$$0.5 \times 0.5 \times 0.5 \times 0.5 = 0.0625$$

As multiplication continues across many layers:

- Gradients become extremely close to zero
- Weight updates become negligible

8.6 Backpropagation and Gradient Shrinking

Backpropagation works layer by layer from the output layer toward earlier layers.

The gradient at each layer depends on:

- The gradient from the next layer
- The derivative of the activation function

For sigmoid activation:

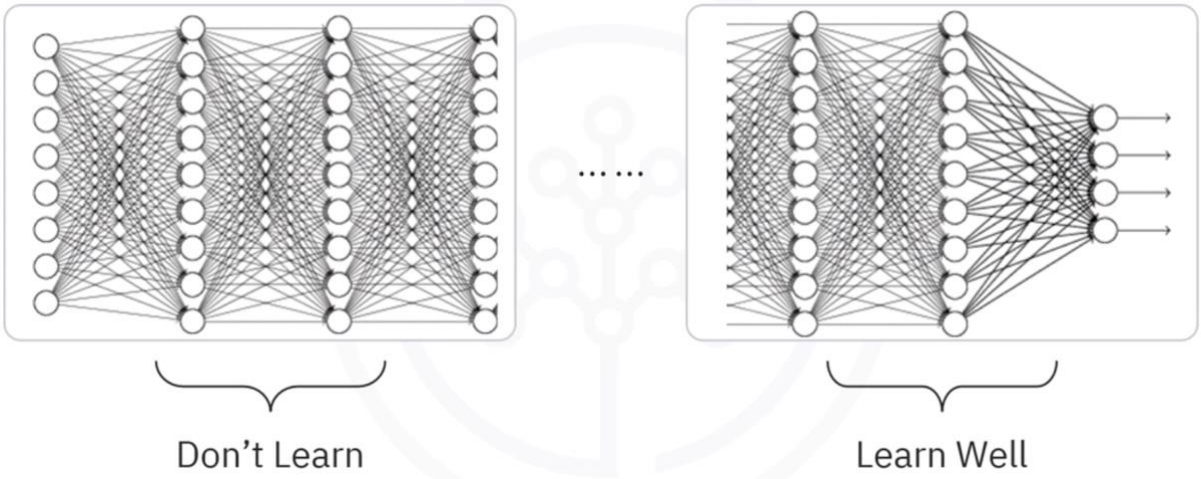
- Derivatives are small
- Repeated multiplication reduces gradients further

8.6.1 Result

Gradients progressively decrease as we move backward through the network.

8.7 Effect on Neural Network Training

8.7.1 Earlier Layers Learn Very Slowly



Because gradients become tiny:

- Weight updates in early layers are extremely small
- Earlier layers learn much more slowly than later layers

8.7.2 Training Becomes Inefficient

This leads to:

- Long training times
- Poor convergence
- Reduced prediction accuracy

8.7.3 Compromised Performance

Since earlier layers fail to learn properly:

- Feature extraction becomes weak
- Overall model performance suffers

8.8 Why Sigmoid Functions Are Avoided

Modern deep learning models generally avoid using sigmoid activation functions in hidden layers because they are prone to vanishing gradients.

8.8.1 Main Reason

Sigmoid functions:

- Produce outputs between 0 and 1
- Cause gradients to shrink during backpropagation
- Slow down learning in deep networks

8.9 Key Characteristics of the Vanishing Gradient Problem

8.9.1 Small Gradients

Gradients become extremely small during backpropagation.

8.9.2 Slow Learning in Early Layers

Neurons in earlier layers update their weights very slowly.

8.9.3 Training Instability

Training deep networks becomes difficult and inefficient.

8.9.4 Reduced Accuracy

The model may fail to learn useful features effectively.

8.9.5 Mathematical Insight

The derivative of the sigmoid activation function is:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

Since:

$$0 < \sigma(x) < 1$$

The derivative is always small.

This contributes directly to shrinking gradients during backpropagation.

8.10 Summary of the Vanishing Gradient Process

- The sigmoid activation function produces outputs between 0 and 1.
- During backpropagation, gradients are multiplied layer by layer.
- Repeated multiplication of small values causes gradients to shrink rapidly.
- Earlier layers receive extremely tiny gradients.
- Weight updates become very small.
- Training becomes slow and less effective.

8.11 Key Takeaways

- The Vanishing Gradient Problem occurs when gradients become extremely small during backpropagation.
- Sigmoid activation functions are highly prone to this problem.
- Repeated multiplication of values less than 1 causes gradients to shrink.
- Earlier layers in deep neural networks learn very slowly.
- Training becomes inefficient, and prediction accuracy decreases.

9 Activation Functions in Neural Networks

9.1 Introduction to Activation Functions

Activation functions play a major role in the learning process of neural networks. They determine whether a neuron should be activated and help neural networks learn complex nonlinear relationships in data.

9.2 Role of Activation Functions

In a neural network:

- Each neuron computes a weighted sum of inputs
- The activation function transforms this value into an output
- This transformation introduces nonlinearity into the network

Without activation functions, neural networks would behave like simple linear models and would not be able to learn complex patterns.

9.3 Types of Activation Functions

There are several activation functions used in neural networks, including:

1. Binary Step Function
2. Linear (Identity) Function
3. Sigmoid (Logistic) Function
4. Hyperbolic Tangent (tanh) Function
5. Rectified Linear Unit (ReLU)
6. Leaky ReLU
7. Softmax Function

The most commonly discussed and widely used activation functions are:

- Sigmoid
- Hyperbolic Tangent (tanh)
- ReLU
- Softmax

9.4 Sigmoid Activation Function

The Sigmoid function is also called the Logistic function.

9.4.1 Formula

$$a = \frac{1}{1 + e^{-z}}$$

9.4.2 Characteristics of the Sigmoid Function

- Output values range between 0 and 1
- At $z = 0$ the output is:

$$a = 0.5$$

- Large positive values of z produce outputs close to 1
- Large negative values of z produce outputs close to 0

9.4.3 Advantages

- Smooth and differentiable
- Produces probability-like outputs
- Historically popular in neural networks

9.4.4 Limitations of the Sigmoid Function

9.4.4.1 Vanishing Gradient Problem

The Sigmoid function becomes flat for large positive or negative values.

Typically:

- Flat near $z > 3$
- Flat near $z < -3$

In these regions:

- Gradients become extremely small
- Weight updates become negligible
- The network learns very slowly or stops learning

This issue is called the vanishing gradient problem.

9.4.4.2 Lack of Symmetry

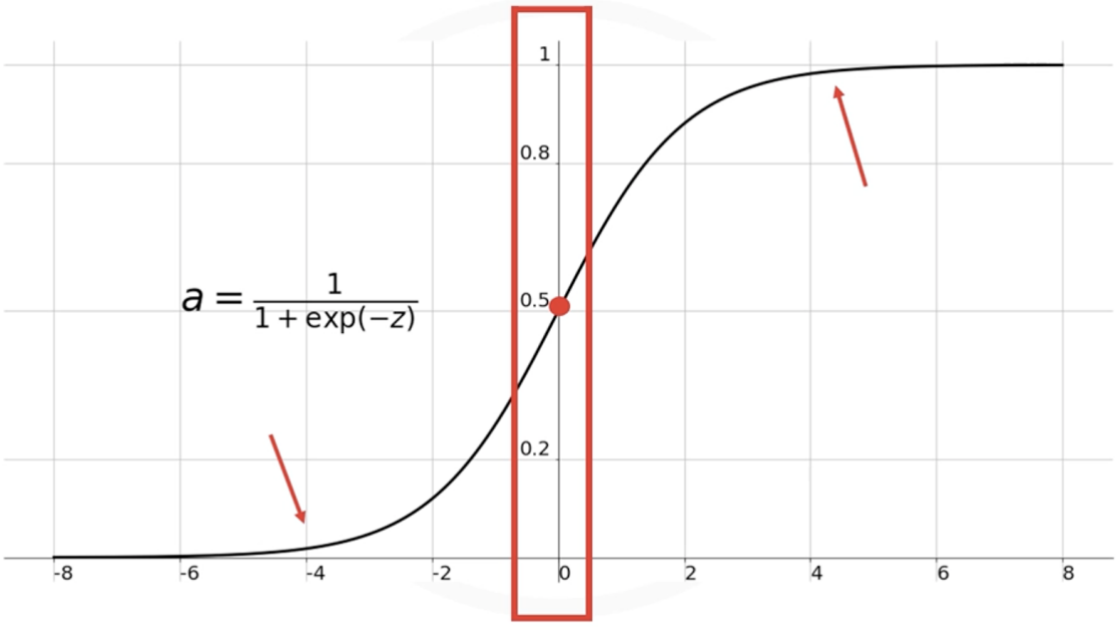
The Sigmoid function outputs values only between:

$$0 \text{ to } 1$$

This means:

- Outputs are always positive
- The function is not symmetric around the origin
- Neurons in the next layer receive activations of the same sign

This can negatively affect optimization.



9.5 Hyperbolic Tangent (tanh) Function

The hyperbolic tangent function is very similar to the Sigmoid function.

9.5.1 Formula

$$a = \tanh(z)$$

9.5.2 Characteristics of tanh

- Output values range between:

$$-1 \text{ to } 1$$

- Symmetrical around the origin
- Scaled version of the Sigmoid function

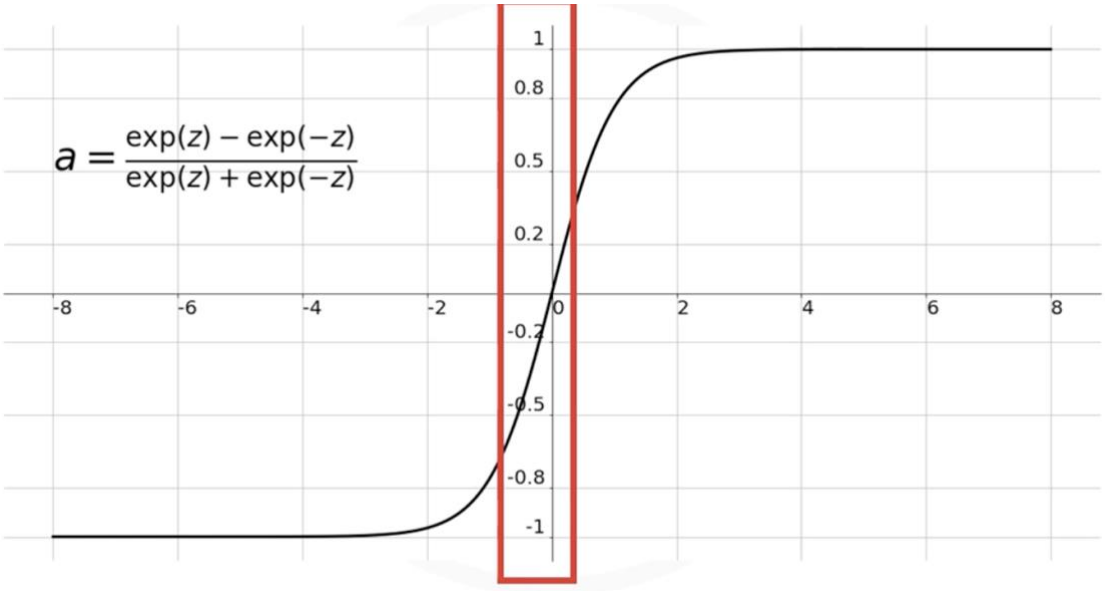
9.5.3 Advantages

- Outputs include both positive and negative values
- Better symmetry improves optimization
- Often performs better than Sigmoid

9.5.4 Limitations

Although tanh solves the symmetry issue, it still suffers from:

- Vanishing gradient problem
- Slow learning in very deep neural networks



9.6 ReLU (Rectified Linear Unit)

The Rectified Linear Unit (ReLU) is the most widely used activation function in modern deep learning.

9.6.1 Formula

$$a = \max(0, z)$$

9.6.2 Behavior of ReLU

- If input is positive → output equals input
- If input is negative → output becomes zero

9.6.3 Example

Input z	Output a
-5	0
-1	0

0	0
3	3
7	7

9.7 Advantages of ReLU

9.7.1 Sparse Activation

ReLU does not activate all neurons simultaneously.

If input is negative:

$$a = 0$$

This means:

- Some neurons remain inactive
- The network becomes sparse
- Computation becomes more efficient

9.7.2 Helps Overcome the Vanishing Gradient Problem

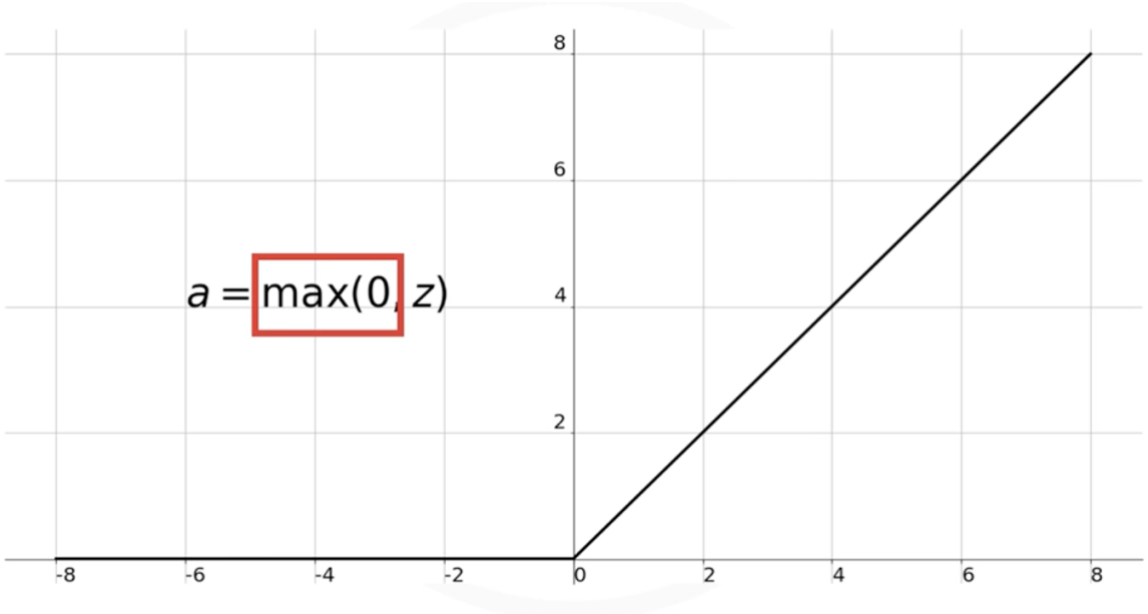
One of the major reasons for ReLU's popularity is that it significantly reduces the vanishing gradient problem.

This advancement played a major role in the success of deep learning.

9.7.3 Computational Efficiency

ReLU is:

- Simple
- Fast to compute
- Efficient for large neural networks



9.8 Leaky ReLU

Leaky ReLU is a variation of ReLU designed to address the “dying ReLU” problem.

Instead of outputting zero for all negative inputs, it outputs a small negative value.

9.8.1 Formula

$$a = \max(0.01z, z)$$

9.8.2 Advantage

- Prevents neurons from permanently becoming inactive
- Maintains small gradients for negative inputs

9.9 Softmax Function

The Softmax function is commonly used in classification problems.

It is typically applied in the output layer of a neural network.

9.9.1 Purpose of Softmax

Softmax converts raw output values into probabilities.

The probabilities:

- Range between 0 and 1
- Sum to 1
- Represent the likelihood of each class

9.9.2 Example

Suppose the output layer produces: [1.6, 0.55, 0.98]

After applying Softmax:

[0.51, 0.18, 0.31]

These values represent class probabilities.

9.9.3 Why Softmax is Useful

Softmax makes it easier to:

- Classify data points
- Determine predicted categories
- Interpret neural network outputs

9.9.4 Softmax Formula

$$a_i = \frac{e^{z_i}}{\sum_{k=1}^{m} e^{z_j}}$$

9.10 Comparison of Common Activation Functions

Activation Function	Output Range	Advantages	Limitations	Common Usage
Sigmoid	0 to 1	Smooth, probabilistic output	Vanishing gradients, not zero-centered	Binary classification output
tanh	-1 to 1	Zero-centered outputs	Vanishing gradients	Sometimes hidden layers
ReLU	0 to ∞	Fast, efficient, reduces vanishing gradients	Can cause dead neurons	Hidden layers
Leaky ReLU	Small negative to ∞	Prevents dead neurons	Slightly more complex	Hidden layers
Softmax	0 to 1 probabilities	Multi-class classification	Used only in output layer	Output layer

9.11 Modern Recommendations

9.11.1 Hidden Layers

The ReLU function is most commonly used in hidden layers because:

- It is computationally efficient
- It reduces vanishing gradients
- It improves deep network training

9.11.2 Output Layer

The activation function used in the output layer depends on the problem type:

Problem Type	Recommended Activation
Binary Classification	Sigmoid
Multi-Class Classification	Softmax
Regression	Linear Activation

9.12 Important Observations

9.12.1 Sigmoid and tanh

These functions:

- Were widely used in older neural networks
- Are avoided in many deep learning applications today
- Can cause vanishing gradient problems

9.12.2 ReLU

ReLU:

- Is the most popular activation function today
- Is mainly used in hidden layers
- Improved deep learning performance significantly

9.12.3 Softmax

Softmax:

- Is mainly used in output layers
- Produces probabilities for classification tasks

9.13 Best Practice for Building Models

When designing a neural network:

1. Start with ReLU in hidden layers
2. Evaluate model performance
3. Switch to other activation functions only if needed

ReLU is usually the default starting choice for deep learning models.

9.14 Key Concepts Summary

9.14.1 Activation Functions

Activation functions:

- Introduce nonlinearity
- Control neuron activation
- Enable neural networks to learn complex patterns

9.14.2 Sigmoid Function

- Outputs values between 0 and 1
- Suffers from vanishing gradients
- Not zero-centered

9.14.3 tanh Function

- Outputs values between -1 and 1
- Symmetric around the origin
- Still suffers from vanishing gradients

9.14.4 ReLU Function

- Most widely used activation function
- Efficient and sparse
- Helps overcome vanishing gradients

9.14.5 Softmax Function

- Converts outputs into probabilities
- Used for classification problems
- Typically placed in the output layer

9.15 Final Takeaways

- Activation functions are essential in neural networks.
- Different activation functions serve different purposes.
- Sigmoid and tanh are less commonly used in deep networks due to vanishing gradients.
- ReLU is the most widely used activation function in hidden layers.
- Softmax is commonly used in output layers for classification tasks.
- Choosing the right activation function significantly impacts neural network performance.

10 Module 2 Summary: Basics of Deep Learning

10.1 Introduction

This module introduced the foundational concepts of Deep Learning, including:

- Gradient Descent
- Learning Rates
- Neural Network Training
- Forward and Backpropagation

- Vanishing Gradient Problem
- Activation Functions

These concepts form the basis of how neural networks learn and optimize their parameters.

10.2 Gradient Descent

Gradient Descent is an iterative optimization algorithm used to find the minimum value of a function.

10.2.1 Key Characteristics

- It minimizes the cost (loss) function
- It updates weights iteratively
- It helps neural networks learn optimal parameters

10.2.2 Gradient Descent Update Rule

$$w_{new} = w_{old} - \eta \frac{dJ}{dw}$$

Where:

- w_{old}= current weight
- w_{new}= updated weight
- η= learning rate
- $\frac{dJ}{dw}$= gradient of the cost function

10.3 Learning Rate

The learning rate controls the size of the update step during optimization.

10.3.1 Large Learning Rate

A large learning rate:

- Produces large update steps
- Can overshoot the minimum point
- May prevent convergence

10.3.2 Small Learning Rate

A small learning rate:

- Produces very small update steps
- Makes training extremely slow
- Requires more iterations to converge

10.3.3 Important Observation

Choosing the proper learning rate is critical for successful training.

10.4 Neural Network Training Process

Neural networks learn by optimizing weights and biases.

10.4.1 Initialization

Initially:

- Weights and biases are assigned random values

10.4.2 Training Loop

The following steps are repeated iteratively:

10.4.2.1 Step 1: Forward Propagation

The network computes predictions using current weights and biases.

10.4.2.2 Step 2: Error Calculation

The predicted output is compared with the ground truth.

The error or loss is computed.

10.4.2.3 Step 3: Backpropagation

Weights and biases are updated using gradients computed from the error.

10.4.2.4 Step 4: Repeat

The process continues until:

- The maximum number of epochs is reached
- Or the error becomes smaller than a predefined threshold

10.5 Forward Propagation

Forward propagation is the process by which input data passes through the network to generate predictions.

10.5.1 Process

- Inputs move layer by layer through the network
- Each neuron performs computations

- Activation functions are applied
- Final predictions are generated at the output layer

10.6 Backpropagation

Backpropagation is the process used to update weights and biases.

10.6.1 Purpose

- Minimize prediction error
- Compute gradients efficiently
- Improve network performance iteratively

10.6.2 Process

- Error gradients are propagated backward through the network
- Gradients determine how weights should change
- Weights are updated using Gradient Descent

10.7 Vanishing Gradient Problem

The vanishing gradient problem is a major issue in deep neural networks.

10.7.1 Main Cause

The problem is mainly caused by sigmoid-like activation functions.

10.7.2 Why It Happens

During backpropagation:

- Gradients are repeatedly multiplied
- These values are often less than one
- Gradients become smaller and smaller as they move backward

As a result:

- Earlier layers receive extremely small gradients
- Learning becomes very slow

10.8 Impact of Vanishing Gradients

10.8.1 Slow Learning in Early Layers

Neurons in earlier layers:

- Learn much more slowly
- Receive tiny gradient updates

10.8.2 Increased Training Time

Because earlier layers train slowly:

- Overall, training takes much longer

10.8.3 Reduced Prediction Accuracy

Poor learning in earlier layers:

- Reduces model effectiveness
- Compromises prediction accuracy

10.9 Why Sigmoid Activation Functions Are Avoided

Sigmoid activation functions are prone to vanishing gradient problems.

Therefore:

- They are rarely used in hidden layers of deep neural networks today

10.10 Activation Functions

Activation functions play a major role in neural network training.

10.10.1 Purpose of Activation Functions

They:

- Introduce non-linearity into the network
- Allow neural networks to learn complex patterns
- Control neuron activation

10.11 Common Activation Functions

10.11.1 Sigmoid Function

The sigmoid function maps values between 0 and 1.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

10.11.2 Characteristics

- Produces probabilities
- Suffers from vanishing gradients
- Not preferred for deep hidden layers

10.12 Hyperbolic Tangent (Tanh) Function

The tanh function is a scaled version of the sigmoid function.

$$tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

10.12.1 Characteristics

- Symmetrical around the origin
- Outputs values between -1 and 1
- Often performs better than sigmoid

10.13 ReLU Function

The Rectified Linear Unit (ReLU) is the most widely used activation function in modern neural networks.

$$ReLU(x) = max(0, x)$$

10.13.1 Advantages of ReLU

- Computationally efficient
- Helps reduce vanishing gradients
- Does not activate all neurons simultaneously
- Enables faster training

10.14 Softmax Function

The Softmax function is commonly used in the output layer of classification networks.

10.14.1 Purpose

It converts outputs into probabilities representing class membership.

10.14.2 Softmax Formula

$$a_i = \frac{e^{z_i}}{\sum_{k=1}^{m} e^{z_j}}$$

10.14.3 Usage

- Multi-class classification problems
- Probability distribution generation
- Final output layer of classifiers

10.15 Key Takeaways

10.15.1 Gradient Descent

- Iterative optimization algorithm
- Used to minimize loss functions

10.15.2 Learning Rate

- Controls update step size
- Must be carefully selected

10.15.3 Neural Network Training

Training involves:

- Forward propagation
- Error calculation
- Backpropagation
- Iterative optimization

10.15.4 Vanishing Gradient Problem

- Mainly caused by sigmoid-like activation functions
- Makes early layers learn slowly
- Increases training time

10.15.5 Activation Functions

Activation functions:

- Add non-linearity
- Improve learning capability
- Influence network performance significantly

10.15.6 ReLU

- Most widely used activation function today
- Efficient and effective for deep networks

10.15.7 Softmax

- Used for classification outputs
- Produces probability distributions for classes

Module 3: Keras and Deep Learning Libraries

11 Deep Learning Libraries

11.1 Introduction to Deep Learning Libraries

Before building deep learning networks, it is important to understand the major deep learning libraries and frameworks available today.

These libraries provide tools and APIs that simplify the process of:

- Building neural networks
- Training deep learning models
- Running computations efficiently on GPUs
- Deploying models into production environments

11.2 Overview of Popular Deep Learning Libraries

The most popular deep learning libraries in descending order are:

1. TensorFlow
2. PyTorch
3. Keras

Another historically important library is:

- Theano

However, Theano is no longer widely used.

11.3 Theano

11.3.1 Introduction

Theano was developed by the Montreal Institute for Learning Algorithms (MILA).

Before TensorFlow and PyTorch became dominant, Theano was one of the major deep learning libraries used for research and development.

11.3.2 Key Features

- Supported deep learning computations
- Enabled GPU acceleration
- Widely used in early deep learning research

11.3.3 Decline in Popularity

Theano gradually lost popularity because:

- The developers could not continuously maintain and support the library
- Newer libraries, such as TensorFlow and PyTorch, have become more advanced and widely adopted

As a result, modern deep learning courses and projects rarely use Theano.

11.4 TensorFlow

11.4.1 Introduction

TensorFlow is the most popular deep learning library for production-level deep learning systems.

It was:

- Developed by Google
- Released publicly in 2015

TensorFlow is still actively used by Google for:

- Research purposes
- Production systems

11.4.2 Popularity of TensorFlow

TensorFlow has a very large community.

Its popularity can be observed through:

- Large number of GitHub forks
- High number of commits
- Many pull requests
- Extensive documentation and tutorials

11.4.2.1 Why TensorFlow is Popular

TensorFlow is widely used because it provides:

- Scalable deep learning solutions
- GPU and TPU support
- Production-ready deployment tools
- Strong ecosystem support

11.4.3 Strengths of TensorFlow

11.4.3.1 Major Advantages

- Highly scalable
- Suitable for production environments
- Strong community support
- Extensive tooling and deployment options
- Backed by Google

11.4.3.2 Common Use Cases

TensorFlow is commonly used for:

- Production AI systems
- Enterprise-level machine learning applications
- Large-scale neural networks
- Cloud deployment

11.4.4 Limitations of TensorFlow

Despite its popularity, TensorFlow:

- Has a steep learning curve
- Can be difficult for beginners
- Requires understanding of complex concepts

11.5 PyTorch

11.5.1 Introduction

PyTorch is another highly popular deep learning library.

It is derived from the Torch framework, which was originally written in Lua.

PyTorch was:

- Released in 2016
- Supported by Facebook (now Meta)

11.5.2 Relationship with Torch

PyTorch is not simply a wrapper around Torch.

Instead:

- It was rewritten specifically for Python
- Designed to feel native and fast
- Optimized for modern deep learning development

11.5.3 Features of PyTorch

PyTorch provides:

- Strong GPU support
- Dynamic computational graphs
- Flexibility for custom deep learning operations
- Easy debugging and experimentation

11.5.4 Growing Popularity of PyTorch

PyTorch has gained immense popularity, especially in:

- Academic research
- Experimental deep learning
- Custom neural network architectures

It is increasingly preferred over TensorFlow for research applications.

11.5.5 Why Researchers Prefer PyTorch

Researchers favor PyTorch because it:

- Offers greater flexibility
- Makes experimentation easier
- Simplifies custom expression optimization
- Provides intuitive debugging

11.5.6 Limitations of PyTorch

Like TensorFlow, PyTorch:

- Has a steep learning curve
- Can be difficult for beginners
- Requires a deeper understanding of neural networks

11.6 Keras

11.6.1 Introduction

Keras is a high-level API for building deep learning models.

It is widely known for:

- Simplicity
- Ease of use
- Fast model development

11.6.2 Main Purpose of Keras

Keras simplifies the process of building deep learning networks.

It allows developers to:

- Build models quickly
- Write less code
- Focus on experimentation rather than implementation complexity

11.6.3 Key Advantages of Keras

11.6.3.1 Ease of Use

Keras is beginner-friendly because of its:

- Clean syntax
- Simple API
- Minimal code requirements

11.6.3.2 Fast Development

Very complex neural networks can be built using only a few lines of code.

11.6.3.3 Rapid Prototyping

Keras is ideal for:

- Beginners
- Educational purposes
- Fast experimentation
- Quick model prototyping

11.6.4 Backend Support in Keras

Keras typically runs on top of low-level deep learning libraries such as TensorFlow.

This means:

- TensorFlow must usually be installed first
- Keras uses TensorFlow as its backend engine

When importing Keras, the backend being used is often displayed explicitly.

11.6.5 Support and Ecosystem

Keras is also supported by Google and integrates closely with TensorFlow.

11.7 TensorFlow vs PyTorch vs Keras

11.7.1 TensorFlow

11.7.1.1 Best For

- Production systems
- Large-scale deployment
- Enterprise AI applications

11.7.1.2 Characteristics

- Powerful
- Scalable
- Complex
- Large ecosystem

11.7.2 PyTorch

11.7.2.1 Best For

- Academic research
- Experimentation
- Custom neural network architectures

11.7.2.2 Characteristics

- Flexible
- Research-friendly
- Dynamic
- Easy experimentation

11.7.3 Keras

11.7.3.1 Best For

- Beginners
- Rapid development
- Quick prototyping

11.7.3.2 Characteristics

- Simple
- High-level
- Easy to learn
- Minimal coding required

11.8 Choosing the Right Library

The choice of library depends on your goals.

11.8.1 Use Keras If

- You are new to deep learning
- You want rapid development
- You prefer simplicity
- You want to build models quickly

11.8.2 Use TensorFlow If

- You need production deployment
- You require scalability
- You want enterprise-level tools

11.8.3 Use PyTorch If

- You are conducting research
- You need flexibility
- You want detailed control over computations
- You are optimizing custom expressions

11.9 Important Takeaways

11.9.1 TensorFlow

- Most popular production deep learning library
- Developed by Google
- Large community and ecosystem
- Widely used in industry

11.9.2 PyTorch

- Based on the Torch framework
- Supported by Meta (Facebook)
- Popular in academic research
- Excellent for custom deep learning applications

11.9.3 Keras

- High-level deep learning API
- Extremely beginner-friendly
- Enables rapid development
- Runs on top of TensorFlow

11.9.4 General Observation

- TensorFlow and PyTorch are powerful but more difficult to learn
- Keras is much easier for beginners and has fast development

11.10 Final Summary

- TensorFlow, PyTorch, and Keras are the most widely used deep learning libraries.
- TensorFlow dominates production-level AI development.
- PyTorch is highly preferred in academic and research environments.
- Keras provides the easiest entry point for beginners.
- Keras simplifies deep learning development using a clean and minimal syntax.
- TensorFlow and PyTorch provide more flexibility and control but require greater expertise.

12 Regression Models with Keras

12.1 Learning Objectives

After reading this topic, we will be able to:

- Prepare data in the correct format for Keras models
- Split a dataset into:
  - Predictor variables
  - Target variable
- Build a simple neural network model using Keras
- Train a regression model using only a few lines of code
- Use the trained model to make predictions

12.2 Introduction to Regression Models in Keras

In this lesson, we learn how to use the Keras library to build deep learning models for regression problems.

A regression problem is a type of machine learning problem where the goal is to predict a continuous numerical value.

12.3 Regression Example: Concrete Compressive Strength Dataset

The example dataset contains information about the compressive strength of concrete samples.

The dataset includes different material quantities used in concrete mixtures, such as:

- Cement
- Blast furnace slag
- Fly ash
- Water
- Superplasticizer
- Coarse aggregate
- Fine aggregate

The target variable is:

- Compressive strength (measured in megapascals)

12.3.1 Example Concrete Sample

	Cement	Blast Furnace Slag	Fly Ash	Water	Superplasticizer	Coarse Aggregate	Fine Aggregate	Age	Strength
0	540.0	0.0	0.0	162.0	2.5	1040.0	676.0	28	79.99
1	540.0	0.0	0.0	162.0	2.5	1055.0	676.0	28	61.89
2	332.5	142.5	0.0	228.0	0.0	932.0	594.0	270	40.27
3	332.5	142.5	0.0	228.0	0.0	932.0	594.0	365	41.05
4	198.6	132.4	0.0	192.0	0.0	978.4	825.5	360	44.30

pandas dataframe (concrete_data)

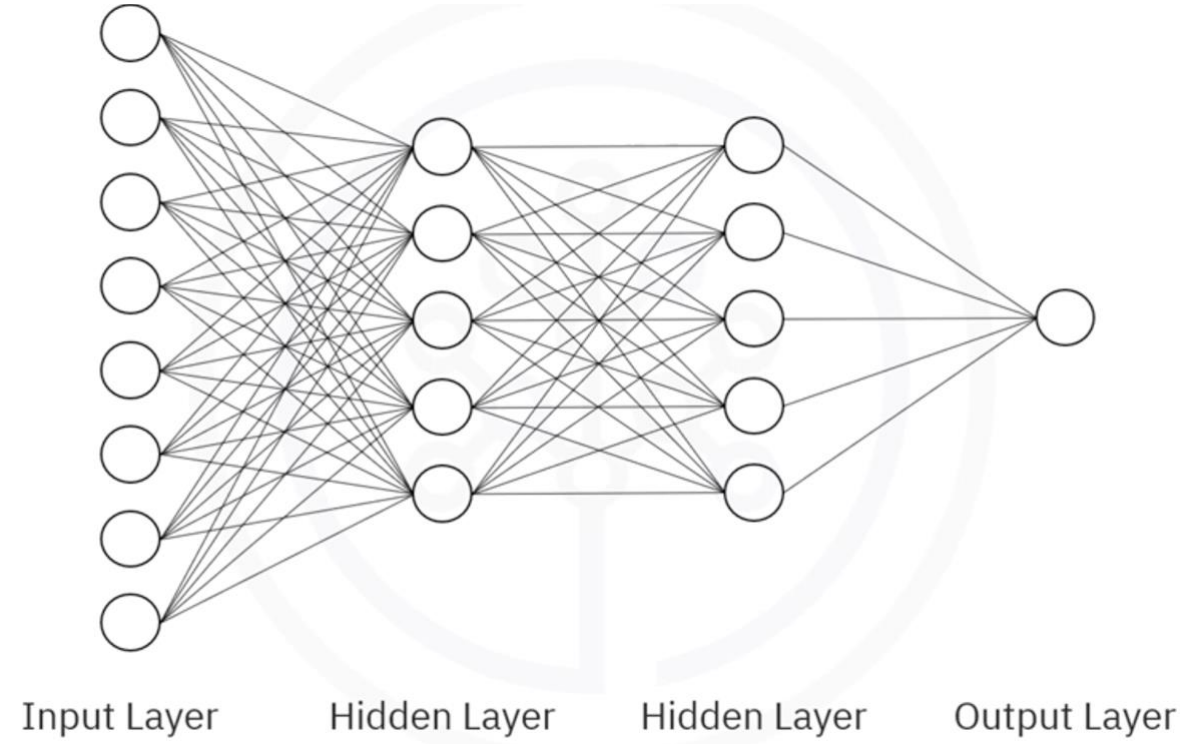
12.4 Goal of the Neural Network

The goal is to create a deep neural network that can automatically predict the compressive strength of concrete based on the ingredient quantities.

12.4.1 Network Architecture

The neural network contains:

- Input Layer:
  - Takes 8 input features
- Hidden Layer 1:
  - 5 neurons
- Hidden Layer 2:
  - 5 neurons
- Output Layer:
  - 1 neuron
  - Outputs predicted compressive strength



12.4.2 Important Note

In real-world applications, hidden layers usually contain many more neurons, such as:

- 50 neurons
- 100 neurons

A smaller network is used here only for simplicity.

12.5 Dense Neural Networks

In this architecture:

- Every neuron in one layer is connected to every neuron in the next layer

Such a network is called a:

12.5.1 Dense Network

Dense layers are among the most commonly used layer types in deep learning.

12.6 Preparing Data for Keras

Before using Keras, the dataset must be prepared correctly.

The dataset is split into:

DataFrame	Purpose
Predictors	Input features
Target	Output variable

12.6.1 Predictor Variables

The predictors contain all input columns.

12.6.2 Target Variable

The target contains the compressive strength column.

12.6.3 Example

```
predictors = concrete_data.drop('Strength', axis=1)
target = concrete_data['Strength']
```

12.7 Importing Keras Libraries

The first step is importing Keras and the Sequential model.

12.7.1 Import Sequential Model

```
from keras.models import Sequential
```

12.8 Sequential Model

The Sequential model is used because the network is a linear stack of layers.

This is the most common type of neural network architecture.

12.8.1 Keras Provides Two Main Model Types

Model Type	Description
Sequential Model	Used for linear stack architectures
Functional API Model	Used for more complex architectures

For most standard problems, the Sequential model is sufficient.

12.9 Creating the Neural Network

To create the model:

```
model = Sequential()
```

12.9.1 Adding Layers to the Model

To create layers, import the Dense layer type.

12.9.1.1 Import Dense Layer

```
from keras.layers import Dense
```

12.9.2 Add Hidden Layers

```
model.add(Dense(5, activation='relu', input_shape=(8,)))
model.add(Dense(5, activation='relu'))
```

12.9.2.1 Explanation

Parameter	Description
5	Number of neurons
activation='relu'	Activation function
input_shape=(8,)	Number of input features

12.9.3 ReLU Activation Function

The ReLU activation function is commonly recommended for hidden layers.

12.9.3.1 Why ReLU?

- Computationally efficient
- Helps reduce vanishing gradient problems
- Works well in deep learning applications

12.9.4 Creating the Output Layer

The output layer contains one neuron because this is a regression problem with one numerical output.

```
model.add(Dense(1))
```

12.9.5 Compiling the Model

Before training, the model must be compiled.

Compilation defines:

- Optimizer
- Loss function

12.9.6 Compile the Model

```
model.compile(optimizer='adam', loss='mean_squared_error')
```

12.9.6.1 Loss Function

The loss function used is:

12.9.6.1.1 Mean Squared Error (MSE)

Mean Squared Error measures the difference between:

- Predicted values
- Actual values

It is one of the most common loss functions for regression problems.

12.9.6.2 Optimizer

The optimizer used is:

12.9.6.2.1 Adam Optimizer

Adam is more efficient than standard gradient descent in many deep learning applications.

12.9.6.2.2 Advantages of Adam

- Faster convergence
- Adaptive learning rates
- No need to manually specify the learning rate

This simplifies model optimization.

12.9.7 Training the Model

The model is trained using the fit() method.

12.9.7.1 Example

```
model.fit(predictors, target, epochs=50)
```

12.9.7.2 Parameters

Parameter	Description
predictors	Input features
target	Ground truth values
epochs	Number of training iterations

12.9.8 Making Predictions

After training, predictions can be generated using the predict() method.

12.9.8.1 Example

```
predictions = model.predict(new_data)
```

12.10 Complete Keras Regression Model Example

```
from keras.models import Sequential
from keras.layers import Dense

# Define predictors and target
predictors = concrete_data.drop('Strength', axis=1)
target = concrete_data['Strength']

# Build model
model = Sequential()

model.add(Dense(5, activation='relu', input_shape=(8,)))
model.add(Dense(5, activation='relu'))
model.add(Dense(1))

# Compile model
model.compile(optimizer='adam', loss='mean_squared_error')

# Train model
model.fit(predictors, target, epochs=50)
```

```
# Make predictions
predictions = model.predict(predictors)
```

12.11 Key Takeaways

- Keras makes building neural networks very simple
- Data must be prepared properly before training
- Regression problems predict continuous numerical values
- Dense networks connect every neuron between layers
- Sequential models are used for linear layer stacks
- ReLU is commonly used for hidden layers
- Mean Squared Error is commonly used for regression loss
- Adam is a powerful and efficient optimizer
- Neural networks in Keras can be built and trained with only a few lines of code

13 Classification Models with Keras

13.1 Introduction to Classification Models

In this lesson, we learn how to use the Keras library to build neural network models for classification problems.

Classification models are used when the target variable belongs to multiple categories or classes instead of predicting a continuous numerical value.

13.1.1 Example Scenario

Suppose we want to build a model that helps determine whether purchasing a particular car is a good decision based on:

- The price of the car
- The maintenance cost
- The number of people the car can accommodate

13.2 Car Dataset Overview

The dataset used in this example is called car_data.

The dataset has already been cleaned and transformed using one-hot encoding.

13.2.1 Features in the Dataset

13.2.1.1 Car Price Categories

The price of the car can be:

- High
- Medium
- Low

13.2.1.2 Maintenance Cost Categories

The maintenance cost can also be:

- High
- Medium
- Low

13.2.1.3 Passenger Capacity

The car can accommodate:

- Two people
- More than two people

13.2.2 Example Data Point

For the first car in the dataset:

- The car is expensive
- Maintenance cost is high
- The car can only fit two people

	price_high	price_low	price_med	maintenance_high	maintenance_low	maintenance_med	persons_2	persons_more	decision
0	1	0	0	1	0	0	1	0	0
1	1	0	0	1	0	0	1	0	0
2	1	0	0	1	0	0	1	0	0
3	1	0	0	1	0	0	1	0	0
4	1	0	0	1	0	0	1	0	0

Pandas dataframe: car_data

$$\text{decision} = \begin{cases} 0 : \text{Not acceptable} \\ 1 : \text{Acceptable} \\ 2 : \text{Good} \\ 3 : \text{Very Good} \end{cases}$$

The target output (decision) is:

Decision Value	Meaning
0	Bad choice / Not acceptable
1	Acceptable
2	Good decision
3	Very good decision

13.3 Neural Network Architecture

The classification model uses a neural network architecture similar to the regression model discussed previously.

13.3.1 Network Structure

- Input Layer:
  - 8 input features (predictors)
- Hidden Layers:
  - 2 hidden layers
  - Each hidden layer contains 5 neurons
- Output Layer:
  - 4 neurons
  - One neuron for each target category

13.4 Predictors and Target

The dataset is divided into:

13.4.1 Predictors

The input variables/features used to make predictions.

Examples:

- Car price
- Maintenance cost
- Passenger capacity

13.4.2 Target

The output variable represents the decision category.

13.5 Transforming the Target Variable

For classification problems in Keras, the target column cannot be used directly.

The target values must first be transformed into binary arrays using one-hot encoding style formatting.

13.5.1 Example Transformation

Original Target	Binary Representation
0	[1, 0, 0, 0]
1	[0, 1, 0, 0]
2	[0, 0, 1, 0]
3	[0, 0, 0, 1]

13.5.2 Using `to_categorical()`

Keras provides the to_categorical() function to perform this transformation.

```
from keras.utils import to_categorical
```

Because the target variable has 4 categories, the output layer must contain 4 neurons.

13.6 Building the Classification Model in Keras

13.6.1 Importing Required Libraries

```
from keras.models import Sequential
from keras.layers import Dense
from keras.utils import to_categorical
```

13.7 Creating the Neural Network

13.7.1 Initializing the Model

```
model = Sequential()
```

13.7.2 Adding Hidden Layers

Two hidden layers are added using the ReLU activation function.

```
model.add(Dense(5, activation='relu', input_shape=(8,)))
model.add(Dense(5, activation='relu'))
```

13.7.3 Adding the Output Layer

The output layer contains 4 neurons and uses the Softmax activation function.

```
model.add(Dense(4, activation='softmax'))
```

13.8 Why Use Softmax?

The Softmax activation function ensures that:

- The output probabilities sum to 1
- Each output neuron represents the probability of belonging to a specific class

This makes Softmax ideal for multi-class classification problems.

13.9 Compiling the Model

For classification problems, we use:

- `categorical_crossentropy` as the loss function
- `accuracy` as the evaluation metric

```
model.compile(
   optimizer='adam',
   loss='categorical_crossentropy',
   metrics=['accuracy']
)
```

13.9.1 Training the Model

The model is trained using the `fit()` method.

This time, the number of epochs is explicitly specified.

```
model.fit(X, y, epochs=50)
```

13.9.2 Epochs

An epoch represents one complete pass of the training dataset through the neural network.

More epochs generally allow the model to learn better patterns from the data.

13.9.3 Making Predictions

Predictions are made using the `predict()` method.

```
predictions = model.predict(X_test)
```

13.9.3.1 Understanding Prediction Output

The output of the prediction method is a probability distribution across all classes.

13.9.3.2 Example Prediction Output

Class	Probability
0	0.99
1	0.01
2	0.00
3	0.00

13.9.3.3 Interpretation

- The probabilities sum to 1
- The class with the highest probability becomes the predicted class
- Higher probabilities indicate higher confidence

13.10 Example Prediction Analysis

First Car

Prediction:

- Probability for class 0 ≈ 0.99

Interpretation:

- The model is highly confident that purchasing the car is not acceptable

13.10.1 Second Car

Prediction:

- Probability for class 0 ≈ 0.99

Interpretation:

- The model again predicts the purchase is not acceptable

13.10.2 Last Three Cars

Prediction:

- Probability for class 1 is slightly higher than class 0

Interpretation:

- The model leans toward accepting the purchase
- However, confidence is lower because probabilities are very close

13.11 Evaluation Metric: Accuracy

Accuracy is used to measure the performance of the classification model.

13.11.1 Accuracy Formula

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

Keras provides accuracy as a built-in evaluation metric.

Custom evaluation metrics can also be defined if needed.

13.12 Key Concepts Learned

13.12.1 Classification Problems

- Classification models predict categories/classes
- Datasets can be split into categories

13.12.2 Dataset Structure

A dataset is divided into:

- Predictors (features)
- Target (output variable)

13.12.3 Target Encoding

For Keras classification problems:

- Target variables must be transformed into binary arrays
- to_categorical() is used for this transformation

13.12.4 Neural Network Design

The classification model contains:

- Input layer
- Hidden layers
- Output layer with Softmax activation

13.12.5 Loss Function

Classification models commonly use:

categorical_crossentropy

instead of:

mean_squared_error

which is typically used for regression problems.

13.12.6 Prediction Output

- Predictions are probabilities
- Probabilities sum to 1
- The highest probability determines the predicted class

13.13 Complete Keras Classification Model Example

```
from keras.models import Sequential
from keras.layers import Dense
from keras.utils import to_categorical

# Convert target column
y = to_categorical(y)

# Build model
model = Sequential()

# Hidden layers
model.add(Dense(5, activation='relu', input_shape=(8,)))
model.add(Dense(5, activation='relu'))

# Output layer
model.add(Dense(4, activation='softmax'))

# Compile model
model.compile(
   optimizer='adam',
   loss='categorical_crossentropy',
   metrics=['accuracy']
)

# Train model
model.fit(X, y, epochs=50)

# Make predictions
predictions = model.predict(X_test)
```

13.14 Summary

In this lesson, we've learned:

- How classification models work in Keras
- How datasets are divided into predictors and targets
- How to convert target values into binary arrays using to_categorical()
- How to build a neural network for classification
- How to use Softmax activation for multi-class outputs
- How to compile, train, and evaluate a classification model
- How to interpret prediction probabilities from the model

14 Module 3 Summary: Keras and Deep Learning Libraries

14.1 Introduction to Deep Learning Libraries

The most popular deep learning libraries are:

- TensorFlow
- PyTorch
- Keras

These libraries are widely used for building, training, and deploying deep learning models.

14.2 TensorFlow

14.2.1 Overview

TensorFlow is one of the most widely used deep learning libraries.

14.2.2 Key Features

- Used extensively in the production of deep learning models
- Supported by a very large developer and research community
- Suitable for large-scale machine learning and deep learning applications

14.3 PyTorch

14.3.1 Overview

PyTorch is based on the Torch framework written in Lua.

14.3.2 Key Features

- Supports machine learning algorithms running on GPUs
- Highly preferred in academic and research environments
- Popular among researchers due to its flexibility and dynamic computation graph

14.3.3 Limitation

- Similar to TensorFlow, PyTorch has a steep learning curve and can be difficult for beginners to use

14.4 Keras

14.4.1 Overview

Keras is a high-level API designed for building deep learning models quickly and easily.

14.4.2 Advantages of Keras

- Easy to use
- Simple and clean syntax
- Facilitates fast development
- Requires only a few lines of code to create complex neural networks

14.4.3 Backend Support

Keras normally runs on top of low-level deep learning libraries such as TensorFlow.

14.5 Data Preparation for Keras

Before using Keras, data must be properly prepared and organized in the correct format.

14.5.1 Dataset Structure

A dataset is typically divided into:

- Predictors (Input Features)
- Target (Output Label)

14.6 Classification Problems in Keras

14.6.1 Target Variable Transformation

When solving classification problems using Keras, the target column must be transformed into an array of binary values.

14.6.2 Using `to_categorical`

The `to_categorical` function from the Keras utilities package is used to convert dataset labels into binary class matrices.

14.6.2.1 Example

```
from tensorflow.keras.utils import to_categorical

binary_targets = to_categorical(target_column)
```

14.7 Building Neural Networks with Keras

Keras allows developers to build and train neural networks with only a few lines of code.

14.7.1 Example Workflow

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential()

model.add(Dense(10, activation='relu', input_shape=(5,)))
model.add(Dense(3, activation='softmax'))

model.compile(optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy'])

model.fit(X_train, y_train, epochs=10)
```

14.8 Key Takeaways

- TensorFlow, PyTorch, and Keras are the most popular deep learning libraries
- TensorFlow is widely used in production environments
- PyTorch is preferred in academic research settings
- Both TensorFlow and PyTorch can be difficult for beginners
- Keras simplifies deep learning development with easy syntax
- Keras can build complex neural networks with very little code
- Proper data preparation is essential before using Keras
- Datasets are divided into predictors and targets
- Classification targets must be converted into binary arrays
- The `to_categorical` function is used for target transformation
- Keras can be used to build classification models efficiently

Module 4: Deep Learning Models

15 Shallow vs Deep Neural Networks

15.1 Introduction

Neural networks can generally be categorized into two types:

- Shallow Neural Networks
- Deep Neural Networks

Understanding shallow neural networks is important because they serve as the foundation for understanding more complex deep neural networks.

15.2 Shallow Neural Networks

15.2.1 Definition

There is no strict universal definition of a shallow neural network, but generally:

- A neural network with **one or two hidden layers** is considered a **shallow neural network**.

15.2.2 Characteristics of Shallow Neural Networks

- Simpler architecture
- Easier to understand and implement
- Typically, take input data in the form of vectors
- Limited ability to automatically extract complex features

15.2.3 Input Type

Shallow neural networks usually require:

- Structured numerical input
- Preprocessed feature vectors

They generally cannot directly process raw data, such as:

- Images
- Audio
- Text

15.3 Deep Neural Networks

15.3.1 Definition

A neural network is considered a deep neural network when it contains:

- Three or more hidden layers
- A large number of neurons in each layer

15.3.2 Characteristics of Deep Neural Networks

- Complex architecture
- Better capability to learn intricate patterns
- Able to automatically extract features from raw data
- Suitable for advanced artificial intelligence applications

15.3.3 Input Type

Deep neural networks can directly process raw data, including:

- Images
- Text
- Audio
- Video

This automatic feature extraction capability is one of the biggest advantages of deep learning.

15.4 Difference Between Shallow and Deep Neural Networks

Feature	Shallow Neural Network	Deep Neural Network
Number of Hidden Layers	1–2 hidden layers	3 or more hidden layers
Complexity	Simpler	More complex
Feature Extraction	Requires manual feature engineering	Automatically extracts features
Input Type	Vector-based input	Raw data input
Learning Capability	Limited for complex tasks	Excellent for complex tasks
Computational Requirement	Lower	Higher

15.5 Reasons Behind the Deep Learning Boom

Deep learning has existed for many years, but its rapid growth and success occurred due to three major factors.

15.6 Advancements in Deep Learning Techniques

15.6.1 ReLU Activation Function

One major breakthrough was the introduction of the ReLU (Rectified Linear Unit) activation function.

Benefits of ReLU:

- Helped overcome the vanishing gradient problem
- Made training deep neural networks easier
- Enabled the creation of very deep architectures

15.6.2 Vanishing Gradient Problem

Earlier activation functions caused gradients to become extremely small during backpropagation, making deep networks difficult to train.

ReLU significantly reduced this issue.

15.7 Availability of Large Amounts of Data

Deep neural networks perform best when trained on massive datasets.

15.7.1 Why Large Data is Important

- Neural networks learn training data extremely well
- Small datasets can lead to overfitting
- More data improves generalization performance

15.7.2 Comparison with Traditional Machine Learning

Traditional machine learning algorithms:

- Improve only up to a certain point with more data
- Eventually, stop gaining significant performance improvements

Deep learning algorithms:

- Continue improving as more data becomes available
- Benefit greatly from massive datasets

15.7.3 Modern Data Availability

Today, huge amounts of data are easily available from:

- Internet platforms
- Social media
- Sensors
- Mobile devices
- Online services

This has accelerated deep learning research and applications.

15.8 Increased Computational Power

15.8.1 Role of GPUs

Modern GPUs (Graphics Processing Units) provide enormous computational power.

Benefits include:

- Faster neural network training
- Ability to process massive datasets
- Reduced training time from weeks to hours

15.8.2 Faster Experimentation

Researchers and developers can now:

- Test multiple architectures quickly
- Experiment with different prototypes

- Improve models in shorter development cycles

15.9 Main Factors Responsible for Deep Learning Growth

The deep learning boom can mainly be attributed to:

- Advancements in deep learning techniques
- Availability of large datasets
- Increased computational power through GPUs

15.10 Key Takeaways

- A neural network with one or two hidden layers is considered shallow.
- A neural network with many hidden layers is considered deep.
- Shallow neural networks primarily take vector inputs.
- Deep neural networks can process raw data such as images and text.
- ReLU activation helped solve the vanishing gradient problem.
- Deep learning benefits significantly from large datasets.
- GPUs made training deep neural networks much faster and more practical.
- These advancements collectively led to the rapid growth of deep learning applications.

16 Convolutional Neural Networks (CNNs)

16.1 What are Convolutional Neural Networks?

Convolutional Neural Networks (CNNs) are a specialized type of neural network designed primarily for processing image data.

CNNs are similar to traditional neural networks because:

- They are composed of neurons
- They use weights and biases
- They optimize parameters during training
- Each neuron computes a dot product between inputs and weights
- Activation functions such as ReLU are commonly used

16.1.1 Difference Between Neural Networks and CNNs

The major difference is that CNNs make the explicit assumption that the inputs are images.

This assumption allows CNNs to:

- Incorporate image-specific properties into the architecture
- Make forward propagation more efficient
- Reduce the number of parameters significantly
- Improve performance for computer vision tasks

16.1.2 Applications of CNNs

CNNs are best suited for:

- Image recognition
- Object detection
- Computer vision applications
- Image classification

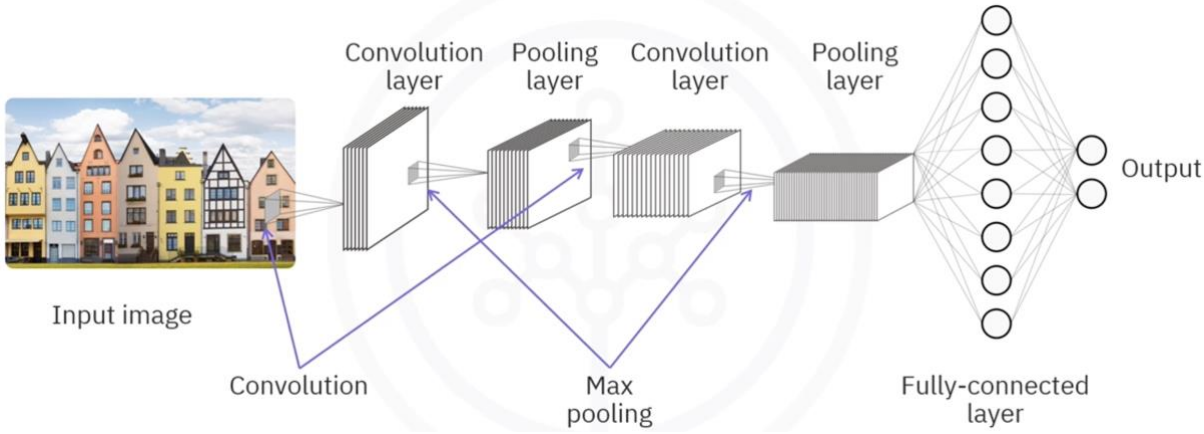
16.2 Architecture of a Convolutional Neural Network

16.2.1 Typical CNN Architecture

A typical convolutional neural network consists of:

1. Convolutional Layers
2. ReLU Activation Layers
3. Pooling Layers
4. Fully Connected Layers
5. Output Layer

These layers work together to extract features from images and perform classification tasks.



16.2.2 Input to a CNN

16.2.2.1 Image Representation

Traditional neural networks usually accept an n-by-1 vector as input.

CNNs, however, accept image data in the following forms:

16.2.2.1.1 Grayscale Images

- Input shape: n-by-m-by-1

16.2.2.1.2 Colored Images

- Input shape: n-by-m-by-3
- The number 3 represents:
    - Red channel
    - Green channel
    - Blue channel

16.2.3 Convolutional Layer

16.2.3.1 Purpose of the Convolutional Layer

The convolutional layer is responsible for extracting features from images.

16.2.3.2 Filters in CNNs

In a convolutional layer:

- Filters (also called kernels) are defined
- Convolution operations are performed between filters and images
- Each filter scans across the image

16.2.3.3 Convolution Process

The convolution process involves:

1. Sliding the filter across the image
2. Computing the dot product between:
    - Filter values
    - Overlapping pixel values
3. Storing the result in an output matrix
4. Repeating the process until the entire image is covered

16.2.3.4 Stride

Stride refers to the number of cells the filter moves each time.

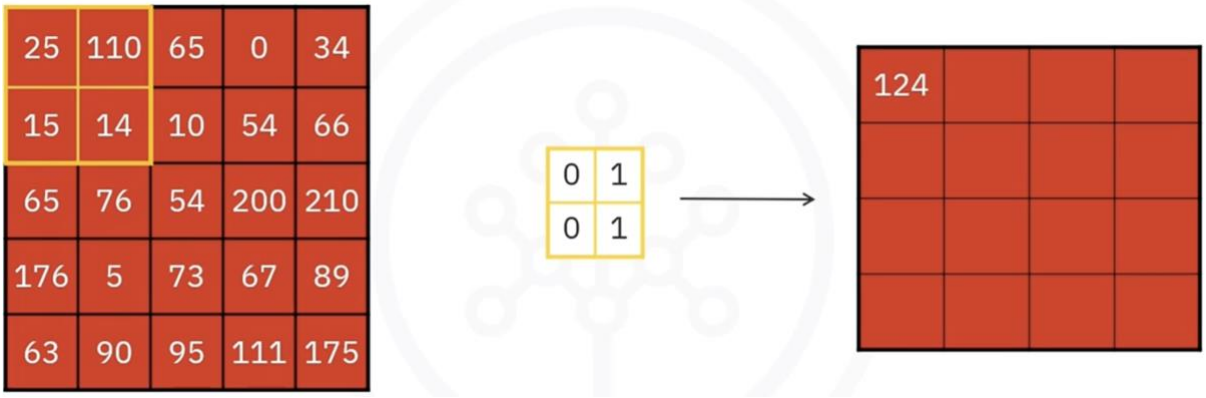
- Stride of 1 → filter moves one cell at a time
- Larger strides reduce output dimensions

16.2.3.5 Multiple Filters

CNNs can use multiple filters.

Advantages of multiple filters:

- Better feature extraction
- Better preservation of spatial information(the information about where things are located and how they are arranged in space(picture or video))
- Ability to learn different image patterns



16.2.3.6 Why Use Convolution?

Instead of flattening(e.g., turning a 28 * 28 picture into a long 1D list like ($28 \times 28 = 784$ values)) images directly into vectors, convolution is used because:

- Flattening creates a massive number of parameters
- Training becomes computationally expensive
- More parameters increase the risk of overfitting
- Convolution reduces parameter count efficiently

16.2.4 ReLU Activation Layer

16.2.4.1 Role of ReLU in CNNs

Convolutional layers are usually followed by ReLU activation functions.

16.2.4.2 ReLU Behavior

- Positive values are preserved
- Negative values are converted to zero

16.2.4.3 Benefits of ReLU

- Introduces non-linearity
- Helps networks learn complex patterns
- Improves training efficiency

16.2.5 Pooling Layer

16.2.5.1 Purpose of Pooling

Pooling layers reduce the spatial dimensions of data propagating through the network.

Benefits include:

- Reduced computational complexity
- Reduced memory usage
- Better generalization
- Reduced overfitting

16.2.5.2 Types of Pooling

16.2.5.2.1 Max Pooling

Max pooling selects the maximum value from each scanned region.

Characteristics:

- Most commonly used pooling method
- Preserves the most important features
- Usually uses larger strides

16.2.5.2.2 Average Pooling

Average pooling computes the average value of each scanned region.

16.2.5.2.3 Spatial Variance

Max pooling provides spatial variance.

This helps CNNs recognize objects even when:

- The object position changes
- The object's appearance slightly varies

16.2.6 Fully Connected Layer

16.2.6.1 Purpose of Fully Connected Layers

The fully connected layer performs the final classification.

16.2.6.2 Flattening

Before entering the fully connected layer:

- The output from previous layers is flattened into a vector

16.2.6.3 Dense Connections

In fully connected layers:

- Every neuron connects to every neuron in the next layer

16.2.7 Output Vector

The output layer generates an n-dimensional vector where:

- n = number of classes

16.2.7.1 Example

For digit classification:

- n = 10
- Represents digits 0 through 9

16.3 Building a CNN Using Keras

16.3.1 Step 1: Create the Model

Use the Sequential constructor to create the model.

```
from tensorflow.keras.models import Sequential

model = Sequential()
```

16.3.2 Step 2: Define Input Shape

For 128 × 128 RGB images:

```
input_shape = (128, 128, 3)
```

16.3.3 Step 3: Add Convolutional Layer

```
from tensorflow.keras.layers import Conv2D

model.add(Conv2D(
   filters=16,
```

```
    kernel_size=(2, 2),
    strides=(1, 1),
    activation='relu',
    input_shape=(128, 128, 3)
))
```

16.3.3.1 Explanation

- 16 filters
- Filter size: 2 × 2
- Stride: 1 × 1
- Activation: ReLU

16.3.4 Step 4: Add Pooling Layer

```
from tensorflow.keras.layers import MaxPooling2D

model.add(MaxPooling2D(
    pool_size=(2, 2),
    strides=2
))
```

16.3.4.1 Explanation

- Pool size: 2 × 2
- Stride: 2
- Uses max pooling

16.3.5 Step 5: Add Additional Convolutional and Pooling Layers

```
model.add(Conv2D(
    filters=32,
    kernel_size=(2, 2),
    activation='relu'
))

model.add(MaxPooling2D(
    pool_size=(2, 2)
))
```

16.3.5.1 Note

The second convolutional layer uses more filters than the first layer.

16.4 Flattening and Fully Connected Layers

16.4.1 Flatten Layer

```
from tensorflow.keras.layers import Flatten

model.add(Flatten())
```

16.4.2 Dense Layer

```
from tensorflow.keras.layers import Dense

model.add(Dense(100, activation='relu'))
```

16.4.3 Output Layer

```
model.add(Dense(num_classes, activation='softmax'))
```

16.4.3.1 Purpose of Softmax

Softmax converts outputs into probabilities.

16.5 Summary

16.5.1 CNN Fundamentals

- CNNs assume that inputs are images
- CNNs are highly effective for computer vision tasks
- CNNs reduce the number of parameters compared to traditional neural networks

16.5.2 CNN Input Shapes

- Grayscale images: n-by-m-by-1
- Colored images: n-by-m-by-3

16.5.3 Convolutional Layer

- Uses filters to extract features
- Performs convolution operations
- Usually followed by ReLU activation

16.5.4 ReLU Activation

- Keeps positive values
- Converts negative values to zero

16.5.5 Pooling Layer

- Reduces spatial dimensions
- Common types:
    - Max pooling
    - Average pooling

16.5.6 Fully Connected Layer

- Flattens extracted features
- Connects neurons densely
- Produces final classification output

16.5.7 Keras Implementation

- CNNs can be built using Keras Sequential models
- Common layers include:
    - Conv2D
    - MaxPooling2D
    - Flatten

- Dense

16.6 Conclusion

Convolutional Neural Networks are powerful deep learning models specifically designed for image processing tasks. By using convolution, pooling, and fully connected layers, CNNs efficiently extract features from images and perform accurate classification and recognition tasks.

17 Recurrent Neural Networks (RNNs)

17.1 Introduction to Recurrent Neural Networks

Recurrent Neural Networks (RNNs) are another type of supervised deep learning model designed to process sequential and time-dependent data.

Traditional neural networks and many deep learning models treat each data point as an independent instance. However, in many real-world applications, data points are connected and depend on previous information.

17.1.1 Problem with Traditional Neural Networks

Traditional neural networks are not suitable for tasks where:

- Data occurs in sequences
- Previous information affects future predictions
- Temporal relationships are important

For example, when analyzing scenes in a movie, the scenes cannot be treated as completely independent because each scene is connected to previous scenes.

Because of this limitation, traditional deep learning models are often unsuitable for sequential data analysis.

17.2 What are Recurrent Neural Networks (RNNs)?

Recurrent Neural Networks (RNNs) are neural networks that contain loops, allowing information from previous inputs to influence future outputs.

Unlike traditional neural networks, RNNs:

- Accept a new input at each time step
- Also use the output from the previous time step as an additional input

This enables the network to remember previous information and learn temporal patterns in data.

17.3 Architecture of a Recurrent Neural Network

The architecture of an RNN works across different time steps.

17.3.1 Time Step t = 0

At the initial time step:

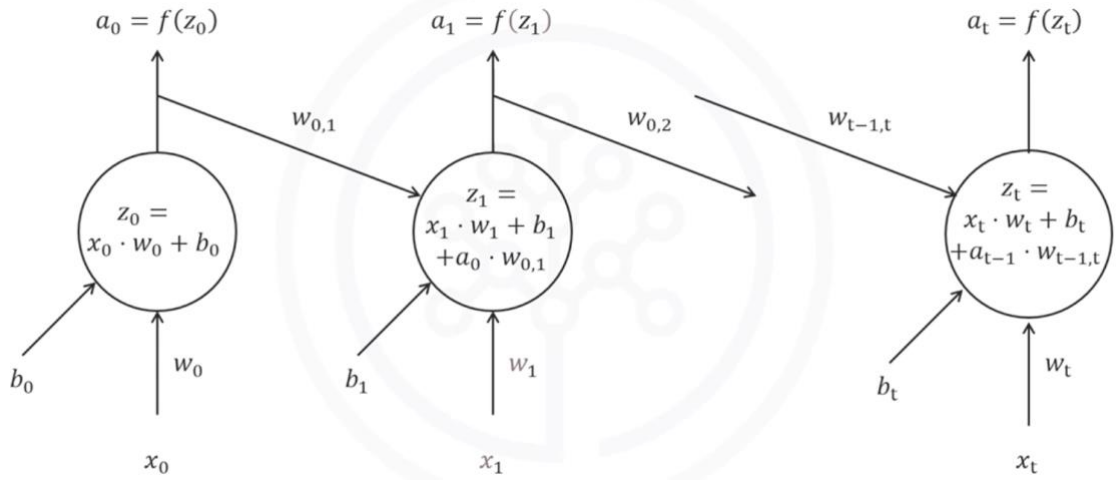
- The network receives input x_0
- The network produces output a_0

17.3.2 Time Step t = 1

At the next time step:

- The network receives a new input x_1
- The previous output $a_0$is also fed back into the network
- The previous output is weighted using weights such as w_{01}

This process continues across multiple time steps, allowing the network to preserve contextual information from earlier inputs.



17.4 Key Feature of RNNs

The major strength of RNNs is their ability to model sequential and temporal data.

RNNs take into account:

- Order of data
- Sequence patterns
- Time dependencies

This gives RNNs a temporal dimension that traditional neural networks lack.

17.5 Applications of Recurrent Neural Networks

RNNs are highly effective for modeling patterns and sequences in data such as:

- Text
- Genomes
- Handwriting
- Stock market data

17.6 Long Short-Term Memory (LSTM)

17.6.1 Introduction to LSTM

A very popular type of recurrent neural network is the Long Short-Term Memory (LSTM) model.

LSTMs are specifically designed to better remember information over long sequences.

17.7 Applications of LSTM Models

LSTM models have been successfully used in many advanced applications.

17.7.1 Image Generation

LSTM-based models can:

- Learn from large image datasets
- Generate completely new images

17.7.2 Handwriting Generation

LSTMs can generate handwriting patterns that resemble human writing.

17.7.3 Automatic Image Captioning

LSTM models can automatically:

- Analyze images
- Generate descriptive captions for images

17.7.4 Automatic Video Description

LSTMs are also used to:

- Understand video sequences
- Automatically describe video content

17.8 Summary

In this lesson, you learned the following concepts:

- Traditional neural networks treat data points as independent instances
- Recurrent Neural Networks (RNNs) use loops to incorporate previous outputs into future computations
- RNNs are designed for sequential and temporal data
- RNNs are effective for applications involving text, genomes, handwriting, and stock market analysis
- Long Short-Term Memory (LSTM) is a popular type of RNN
- LSTMs are widely used for image generation, handwriting generation, automatic image captioning, and video description systems

18 Transformers

18.1 Introduction to Transformers

Transformers are a type of neural network architecture that has revolutionized natural language processing (NLP) and beyond. They are widely used in deep learning models that power AI tools such as ChatGPT and Gemini.

Unlike traditional recurrent neural networks (RNNs) and convolutional neural networks (CNNs), transformers excel at capturing long-range dependencies in sequential data. This makes them highly effective for tasks such as:

- Machine translation
- Text summarization
- Question answering
- Text-to-image generation

18.2 Key Applications and Transformer Models

Different types of transformer-based models include:

- Generative pre-trained models used in ChatGPT
- Bidirectional transformers (BERT) are used in Google Search and Google Translate
- Image transformers used in applications such as Adobe Photoshop

18.3 Attention Mechanism Overview

The key innovation in transformers is the attention mechanism.

This mechanism allows the model to:

- Weigh the importance of different parts of an input sequence
- Focus on relevant tokens while processing data
- Capture long-range dependencies effectively

18.4 Self-Attention Mechanism in Transformers

The self-attention mechanism is used to process textual data and consists of three main components:

18.4.1 1. Query, Key, and Value (Q, K, V)

For each input token, three vectors are generated:

- **Query (Q):** Represents the word we are focusing on
- **Key (K):** Represents all other words in the sequence
- **Value (V):** Contains the information passed to the next layer

18.4.2 2. Attention Scores

- The query vector of a token is compared with the key vectors of all other tokens using a dot product
- This produces attention scores that indicate how relevant each token is to the current token

18.4.3 3. Weighted Sum (Context Vector)

- Attention scores are normalized (using softmax)
- A weighted sum of value vectors is computed
- This produces a context-aware representation of each token

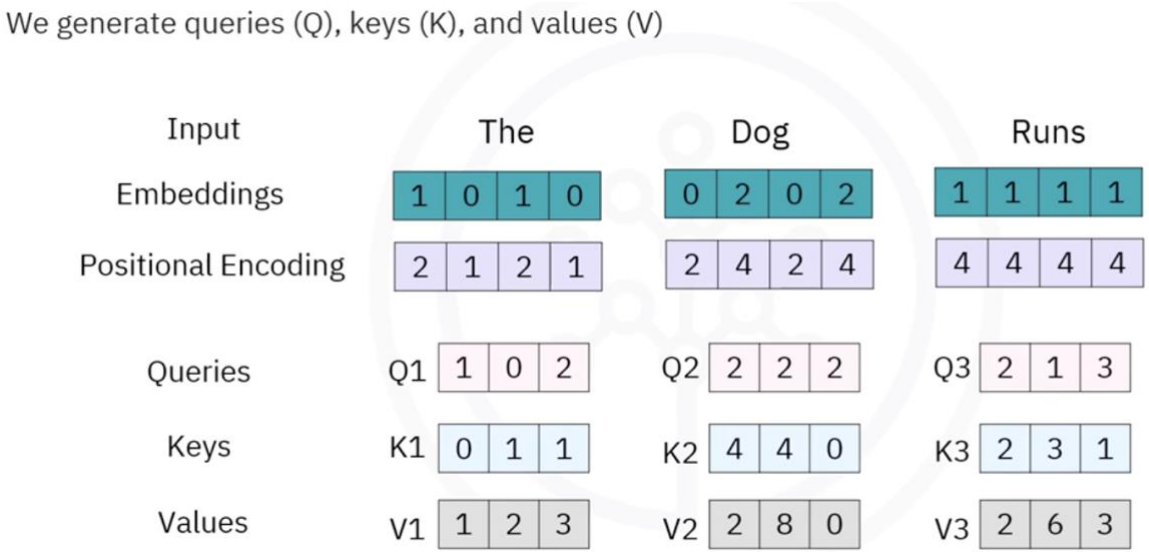
18.5 Example: Self-Attention with “The Dog Runs”

To understand self-attention:

- Input sentence: “The dog runs.”
- Each word is converted into an embedding vector
- Positional encoding is added to preserve word order
- Q, K, and V vectors are generated for each word

18.5.1 Attention Computation Process

- Compute the dot product between the query and key vectors
- Form an attention score matrix
- Scale scores and apply softmax to convert them into probabilities
- Compute the weighted sum of value vectors



18.5.2 Result

- Each word becomes contextually enriched
- The output is a new contextualized vector representation of the sentence

18.6 Cross-Attention Mechanism (Text-to-Image Generation)

Transformers are also used for text-to-image generation through cross-attention.

18.6.1 What is Cross-Attention?

Cross-attention allows one type of data (e.g., text) to influence the generation of another type of data (e.g., images).

18.6.2 Process in Text-to-Image Models

Given a prompt such as:
"A two-story house with a red roof and a garden in front"

The process is as follows:

- The text is processed using self-attention to produce contextual embeddings(Numerical representations) because transformers don't understand words like humans.
- These embeddings are passed through a transformer encoder to generate query representations (Q)
- An image generation model (e.g., DALL·E) uses cross-attention based on these queries
- The model generates images autoregressively, predicting one part at a time

18.6.3 Key Idea

- The model does not retrieve an existing image
- Instead, it synthesizes a new image based on learned understanding

- It can create novel and imaginative outputs (e.g., a turtle driving a car, or a horse with bamboo-like legs)
- Multiple variations of the same prompt can be generated

18.6.4 Contextual Embeddings

A neural network cannot directly understand words like humans do.

So each word is converted into a vector (a list of numbers).

Example:

"cat" → [0.21, -0.44, 0.91, ...]

Transformers solve this problem using:

18.6.4.1 Contextual Embeddings

Now the embedding depends on the surrounding words.

So:

bank(finance) ≠ bank(river)

The transformer creates different representations based on context.

18.6.4.2 Example

Sentence 1

"I deposited money in the bank."

The word **bank** becomes associated with:

- money
- deposited
- finance

Sentence 2

"The fisherman sat on the bank."

Now, the **bank** becomes associated with:

- fisherman
- river
- water

So the embedding changes dynamically.

18.7 Advantages of Transformers Compared to RNNs

Transformers offer several advantages over RNNs:

- Parallel processing of data significantly speeds up training
- Better at capturing long-range dependencies
- Avoid issues like vanishing gradients
- More effective for large-scale NLP tasks

18.7.1 RNN Limitations

- Process data sequentially (slow training)
- Struggle with long-term dependencies
- Limited ability to scale efficiently

18.8 Limitations of Transformers

Despite their success, transformers have some limitations:

- Require very large datasets for training
- High computational and memory requirements
- Inherent biases present in training data
- May struggle with generalization outside training distribution

18.9 Summary and Key Takeaways

Transformers are a powerful neural network architecture that has transformed modern AI and NLP.

Key points:

- They are based on the attention mechanism
- Self-attention uses Query, Key, and Value vectors to compute context
- Cross-attention enables text-to-image generation
- They outperform RNNs in scalability and performance
- They are widely used in applications like translation, summarization, and generative AI
- They require large datasets and can inherit bias from the data

Transformers remain one of the most important breakthroughs in modern deep learning and are central to many AI systems used today.

19 Autoencoders

19.1 Introduction to Autoencoders

So far, two supervised deep learning models have been discussed:

- Convolutional Neural Networks (CNNs)

- Recurrent Neural Networks (RNNs)

Autoencoders are different because they belong to the category of **unsupervised deep learning models**.

19.2 What Are Autoencoders?

Autoencoding is a **data compression algorithm** in which:

- The compression function is learned automatically from data
- The decompression function is also learned automatically from data
- No manual engineering by humans is required

Autoencoders are built using **neural networks**.

19.3 Characteristics of Autoencoders

19.3.1 Data-Specific Nature

Autoencoders are **data-specific**, meaning they only perform well on data similar to the data they were trained on.

For example:

- An autoencoder trained on images of cars will effectively compress car images
- The same autoencoder will perform poorly on images of buildings because it has learned vehicle-specific features

19.4 Applications of Autoencoders

Autoencoders have several important applications, including:

- Data denoising
- Dimensionality reduction
- Data visualization

19.5 Architecture of an Autoencoder

An autoencoder typically consists of two major components:

19.5.1 Encoder

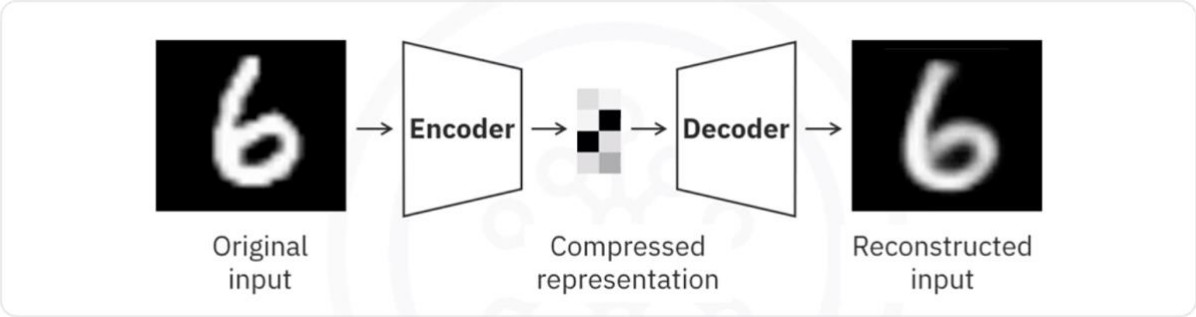
The encoder:

- Accepts the input data (such as an image)
- Learns the optimal compressed representation of the input

19.5.2 Decoder

The decoder:

- Uses the compressed representation
- Reconstructs or restores the original input image



19.6 How Autoencoders Work

Autoencoders are unsupervised neural networks that use **backpropagation** during training.

19.6.1 Key Idea

The target output is set to be the same as the input.

In other words, the network attempts to learn an approximation of the identity function.

19.6.2 Importance of Nonlinear Activation Functions

Because neural networks use nonlinear activation functions, autoencoders can learn complex data projections.

This makes them more powerful than techniques such as:

- Principal Component Analysis (PCA)

PCA and similar traditional methods can only handle **linear transformations**, whereas autoencoders can capture nonlinear relationships in data.

19.7 Restricted Boltzmann Machines (RBMs)

19.7.1 Definition

Restricted Boltzmann Machines (RBMs) are a popular type of autoencoder.

RBMs have been successfully applied in several machine learning tasks.

19.8 Applications of Restricted Boltzmann Machines

19.8.1 Fixing Imbalanced Data Sets

RBMs can help balance imbalanced datasets because they learn the distribution of the input data.

They can:

- Learn the characteristics of the minority class
- Generate additional samples for the minority class
- Transform an imbalanced dataset into a balanced dataset

19.8.2 Estimating Missing Values

RBMs can estimate missing values in different features of a dataset.

19.8.3 Automatic Feature Extraction

RBMs are also widely used for:

- Automatic feature extraction
- Especially for unstructured data

19.9 Summary

19.9.1 Autoencoders

- Autoencoding is a data compression algorithm
- Compression and decompression functions are learned automatically from data
- Autoencoders are data-specific
- Applications include:
    - Data denoising
    - Dimensionality reduction
    - Data visualization

19.9.2 Autoencoder Workflow

An autoencoder:

1. Takes an input (such as an image)
2. Uses an encoder to create a compressed representation
3. Uses a decoder to reconstruct the original input

19.9.3 Restricted Boltzmann Machines (RBMs)

- RBMs are a popular type of autoencoder
- Applications include:
    - Fixing imbalanced datasets

- Estimating missing values
- Automatic feature extraction for unstructured data

20 Using Pretrained Models as Feature Extractors in Keras

20.1 Introduction to Pretrained Models

Pretrained models are neural network models that have already been trained on very large datasets such as ImageNet. These models learn useful and general-purpose features that can later be reused for new tasks.

Instead of training a deep neural network from scratch, we can reuse these pretrained models to extract meaningful features from new data.

Common pretrained models include:

- VGG16
- ResNet

These models are widely used because they can capture rich visual patterns such as:

- Edges
- Shapes
- Textures
- Object parts
- High-level image representations

20.2 Using Pretrained Models as Feature Extractors

Using a pretrained model as a feature extractor means:

- Keeping the pretrained weights unchanged
- Using the model to extract high-level features from images
- Applying these extracted features to downstream tasks

The pretrained model acts as a fixed feature extractor without additional retraining.

20.2.1 Common Downstream Tasks

Extracted features can be used for:

- Clustering
- Visualization
- Dimensionality Reduction
- Image Classification
- Feeding features into traditional machine learning models

20.3 Benefits of Using Pretrained Models

20.3.1 No Additional Training Required

The pretrained model is used directly without retraining.

Benefits include:

- Faster implementation
- Reduced training time
- Lower computational cost

20.3.2 Efficient Feature Extraction

The convolutional layers in pretrained models learn hierarchical representations of images.

These learned representations are highly effective for:

- Feature extraction
- Pattern recognition
- Visual understanding tasks

20.3.3 Suitable for Limited Resources

This approach is useful when:

- Computational resources are limited
- The dataset is small
- Training a deep neural network from scratch is impractical

20.4 Example: Using VGG16 for Feature Extraction in Keras

This example demonstrates how to use a pretrained model in Keras for feature extraction.

20.4.1 Step 1: Create Sample Dataset

The following steps are used to generate sample image data:

- Create directories for two classes:
    - Class One
    - Class Two
- Define a function to generate random images
- Save generated images into their respective directories
- Use these images as input data for the pretrained model

20.4.2 Step 2: Load the Pretrained Model

Load the pretrained VGG16 model trained on ImageNet.

Key configuration:

- Exclude the top fully connected layers
- Use an input shape of:

(224, 224, 3)

This allows the model to act purely as a feature extractor.

20.5 Model Architecture

20.5.1 Flatten Layer

The Flatten() layer converts multidimensional feature maps into a one-dimensional vector.

```
Flatten()
```

20.5.2 Dense Layer

Adds a fully connected layer with:

- 256 units
- ReLU activation

```
Dense(256, activation='relu')
```

20.5.3 Output Layer

Adds a final output layer with:

- 1 unit
- Sigmoid activation

```
Dense(1, activation='sigmoid')
```

20.6 Model Compilation

The model is compiled using:

- Adam optimizer
- Binary cross-entropy loss function

```
model.compile(
   optimizer='adam',
   loss='binary_crossentropy',
   metrics=['accuracy']
)
```

20.7 Image Preprocessing

20.7.1 ImageDataGenerator

ImageDataGenerator rescales pixel values from:

0–255 → 0–1

Example:

```
ImageDataGenerator(rescale=1./255)
```

20.8 Loading Data Using flow_from_directory

The flow_from_directory() method:

- Loads training images from directories
- Resizes images to 224 × 224
- Processes images in batches
- Assigns binary labels

Example configuration:

```
train_generator.flow_from_directory(
   'sample_data',
   target_size=(224,                                                    224),
   batch_size=32,
   class_mode='binary'
)
```

20.9 Training the Model

The model is trained using:

```
model.fit(train_generator, epochs=10)
```

20.9.1 Training Details

- The dataset is processed in batches
- Training occurs for 10 epochs
- Extracted features are used for classification

20.10 Fine-Tuning Pretrained Models

Fine-tuning is an advanced form of transfer learning.

It involves:

- Unfreezing some top layers of the pretrained model
- Training these layers together with the newly added layers

This allows the model to better adapt to the new dataset.

20.11 Transfer Learning

Transfer learning refers to adapting a pretrained model for a new but related task.

Benefits include:

- Better performance
- Reduced training data requirements
- Faster convergence
- Improved accuracy

This is especially useful when:

- The new dataset is small
- The new task is related to the original task
- Training from scratch is expensive

20.12 How Fine-Tuning Improves Performance

Fine-tuning helps the model:

- Adjust pretrained weights
- Learn task-specific features
- Improve generalization on new data

Performance improvements are significant when:

- The new dataset differs from the original dataset
- The new task is similar but not identical

20.13 Key Takeaways

20.13.1 Pretrained Models

Pretrained models provide powerful feature extraction capabilities learned from large datasets.

20.13.2 Feature Extraction

You can use pretrained models directly without retraining to extract useful image features.

20.13.3 Fine-Tuning

Fine-tuning improves performance by adapting pretrained weights to a new task.

20.13.4 Transfer Learning

Transfer learning enables high performance even when limited training data is available.

20.13.5 Advantages

Using pretrained models offers:

- Faster development
- Lower computational requirements

- Better accuracy with smaller datasets
- Reduced training time

20.14 Summary

In this lesson, you learned:

- What pretrained models are
- How to use pretrained models as feature extractors
- How feature extraction works in Keras
- How to use VGG16 for downstream tasks
- The role of transfer learning and fine-tuning
- The advantages of leveraging pretrained models trained on ImageNet

By leveraging pretrained models, you can significantly improve model performance while reducing the amount of required training data and computational resources.

21 Module 4 Summary: Deep Learning Models

21.1 Introduction

This module covered the fundamentals of deep learning models, including neural networks, convolutional neural networks (CNNs), recurrent neural networks (RNNs), long short-term memory networks (LSTMs), and autoencoders. It also explored their architectures, working principles, and practical applications.

21.2 Shallow Neural Networks vs Deep Neural Networks

21.2.1 Shallow Neural Networks

- A neural network with only **one hidden layer** is called a **shallow neural network**.
- Shallow neural networks generally accept only **vector inputs**.

21.2.2 Deep Neural Networks

- A neural network with **many hidden layers** and a **large number of neurons** in each layer is called a **deep neural network**.
- Deep neural networks can directly process **raw data**, including:
    - Images
    - Text
    - Audio

21.2.3 Reasons for the Growth of Deep Learning

The rapid advancement of deep learning can be attributed to three major factors:

- Advancements in deep learning research
- Availability of massive datasets
- Increased computational power

21.3 Convolutional Neural Networks (CNNs)

21.3.1 Overview

Convolutional Neural Networks (CNNs) are specialized neural networks designed primarily for image-related tasks.

CNNs assume that the input data consists of images.

21.3.2 Applications of CNNs

CNNs are highly effective for:

- Image recognition
- Object detection
- Computer vision applications

21.3.3 Input Structure in CNNs

The input to a CNN is typically:

- $(n \times m \times 1)$ for grayscale images
- $(n \times m \times 3)$ for colored images

Where:

- n = image height
- m = image width
- 1 or 3 = number of color channels

21.4 Convolutional Layer

21.4.1 Filters and Convolution

- In the convolutional layer, filters are defined.
- Convolution operations are performed between the filters and the image data.

21.4.2 ReLU Activation Function

CNNs commonly use the ReLU activation function.

21.4.2.1 Purpose of ReLU

- Passes positive values
- Converts negative values to 0

This helps introduce non-linearity into the model.

21.5 Pooling Layer

21.5.1 Purpose of Pooling

Pooling layers reduce the spatial dimensions of the data flowing through the network.

Benefits include:

- Reduced computational cost
- Reduced overfitting
- Faster processing

21.5.2 Types of Pooling

21.5.2.1 Max Pooling

- Selects the maximum value from a region

21.5.2.2 Average Pooling

- Computes the average value from a region

21.6 Fully Connected Layer

21.6.1 Flattening

- The output from the final convolutional layer is flattened into a vector.

21.6.2 Dense Connections

- Every node in the current layer is connected to every node in the next layer.

This layer is mainly responsible for final prediction and classification tasks.

21.7 Recurrent Neural Networks (RNNs)

21.7.1 Overview

Traditional neural networks treat data points as independent instances.

RNNs differ because they:

- Take the current input
- Also use the output from the previous time step

This allows them to model sequential data effectively.

21.7.2 Applications of RNNs

RNNs are useful for:

- Text processing
- Genome analysis
- Handwriting recognition
- Stock market prediction

21.8 Long Short-Term Memory Networks (LSTMs)

21.8.1 Overview

LSTMs are a popular type of RNN designed to better handle long-term dependencies.

21.8.2 Applications of LSTMs

LSTMs are commonly used in:

- Image generation
- Handwriting generation
- Automatic image captioning
- Automatic video description generation

21.9 Autoencoders

21.9.1 Overview

Autoencoding is a data compression technique where:

- Compression and decompression functions are automatically learned from data.

21.9.2 Characteristics of Autoencoders

- Autoencoders are highly data-specific.
- They learn efficient representations of input data.

21.9.3 Applications of Autoencoders

Common applications include:

- Data denoising
- Dimensionality reduction
- Data visualization

21.10 Autoencoder Architecture

21.10.1 Encoder

- Compresses the input into an optimal lower-dimensional representation.

21.10.2 Decoder

- Reconstructs the original input from the compressed representation.

21.10.3 Example Workflow

For image data:

1. The image is provided as input.
2. The encoder compresses the image.
3. The decoder reconstructs the original image.

21.11 Restricted Boltzmann Machines (RBMs)

21.11.1 Overview

Restricted Boltzmann Machines are a popular type of autoencoder-related model.

21.11.2 Applications of RBMs

RBMs are useful for:

- Handling imbalanced datasets
- Estimating missing dataset values
- Automatic feature extraction

21.12 Key Takeaways

- Shallow neural networks contain one hidden layer, while deep neural networks contain many hidden layers.
- CNNs are designed for image processing tasks.
- Pooling layers reduce dimensionality and computational complexity.
- RNNs and LSTMs are ideal for sequential and time-dependent data.
- Autoencoders learn compressed representations of data.
- RBMs help with feature extraction and handling incomplete datasets.